

# ME5412 Robotics for Healthcare

## Interaction Control

Cecilia Laschi

Department of Mechanical Engineering, NUS

[cecilia.laschi@nus.edu.sg](mailto:cecilia.laschi@nus.edu.sg)

# Outline

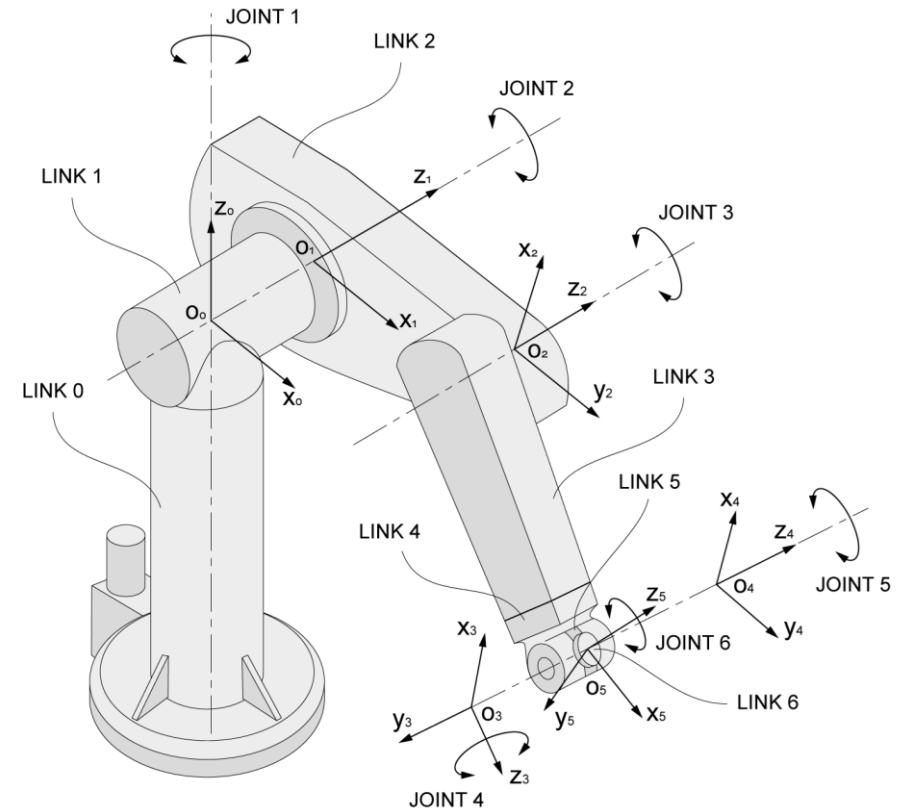
- Recalling robot control basics
- Introducing external forces in the robot dynamic model
- Techniques for controlling interaction forces

# A Classical Robotic Arm

- A chain of rigid links
- The base is fixed, the free end is the end effector
- Links connected with revolute joints
  - Anthropomorphic arm
- Rotational joint (revolute)
  - The relative position of 2 links is fully described by an angle  $\theta$

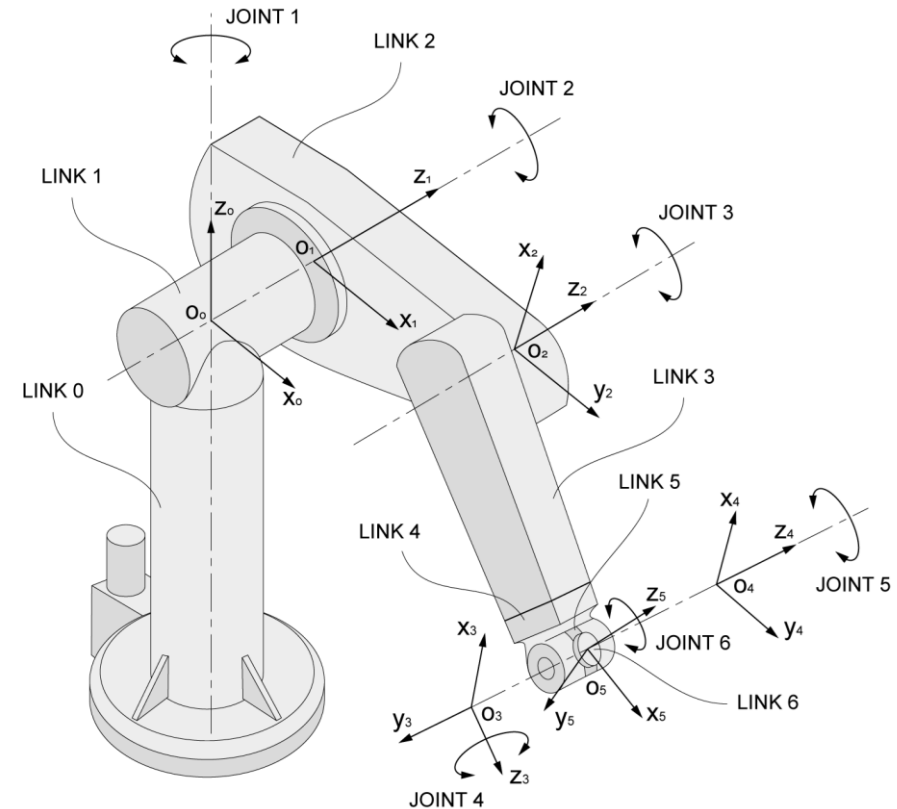
Other joint type:

- Translational joint (prismatic)
  - The relative position of 2 links is fully described by a distance  $d$
- If all prismatic joints
  - Cartesian robot



# Degree of Freedom (DOF)

- The **degree of freedom (DOF)** is any of the minimum number of coordinates required to completely specify the motion of a mechanical system.
- In a robot like the one in the figure, the number of link-joint pairs determines the number of DOFs
- We assume we have an **actuator** (e.g., motor) on each joint and that the rotation axes of the joint and the motor coincide.



# Joint Space and Cartesian Space

- **Joint space** (or configuration space) is the space in which the  $\mathbf{q}$  vector of joint variables is defined.

- The joint space dimension is indicated with  $n$  ( $n$  = number of joints in the robot).

$$\mathbf{q} = \begin{bmatrix} q_0 \\ \vdots \\ q_{n-1} \end{bmatrix}$$

- **Cartesian space** (or task space) is the space in which the  $\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \Phi \end{bmatrix}$  vector of the end-effector position is defined.

- The Cartesian space dimension is indicated with  $m$  ( $m = ?$ ).

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \Phi \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \psi \end{bmatrix}$$

# Joint Space and Cartesian Space

- **Joint space** (or configuration space) is the space in which the  $\mathbf{q}$  vector of joint variables is defined.

- The joint space dimension is indicated with  $n$  ( $n$  = number of joints in the robot).

$$\mathbf{q} = \begin{bmatrix} q_0 \\ \vdots \\ q_{n-1} \end{bmatrix}$$

- **Cartesian space** (or task space) is the space in which the  $\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \Phi \end{bmatrix}$  vector of the end-effector position is defined.

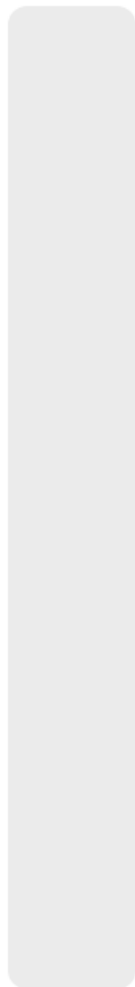
- The Cartesian space dimension is indicated with  $m$  ( $m = 6$ ).

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \Phi \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \psi \end{bmatrix}$$

What is the relation between  $q$  and  $x$ ? Can we find  $x$  given  $q$  and  $q$  given  $x$ ?

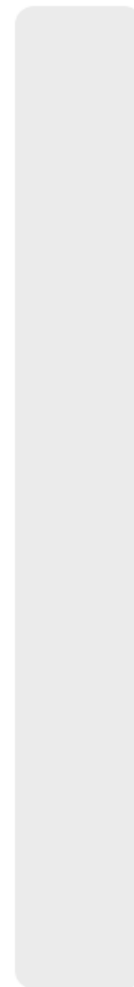


0%



Yes

0%



No

# Forward (or Direct) Kinematics

- Given a position  $\mathbf{q}$  in joint space, the corresponding position in Cartesian space  $\mathbf{x}$  is uniquely determined
- The function to obtain  $\mathbf{x}$  from  $\mathbf{q}$  is the forward kinematic transformation  $K$ :

$$\mathbf{x} = K(\mathbf{q})$$

## Inverse Kinematics

- Given a desired position  $\mathbf{x}$  in Cartesian space, the corresponding position in joint space is given by the inverse kinematic transformation  $K^{-1}$ :

$$\mathbf{q} = K^{-1}(\mathbf{x})$$

# Differential Kinematics

- Transformations in the velocity space.
- **Forward differential kinematics**, from joint velocity to end-effector velocity:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$$

- **Inverse differential kinematics**, from end-effector velocity to joint velocity:

$$\dot{\mathbf{q}} = \mathbf{J}^{-1}(\mathbf{q})\dot{\mathbf{x}}$$

- The transformation matrix is the Jacobian matrix  $\mathbf{J}(\mathbf{q})$
- The Jacobian matrix is not composed of geometric, constant parameters but depends on the current robot configuration  $\mathbf{q}$ .

## Familiar with PID control?



Of course

0%

Basic principle

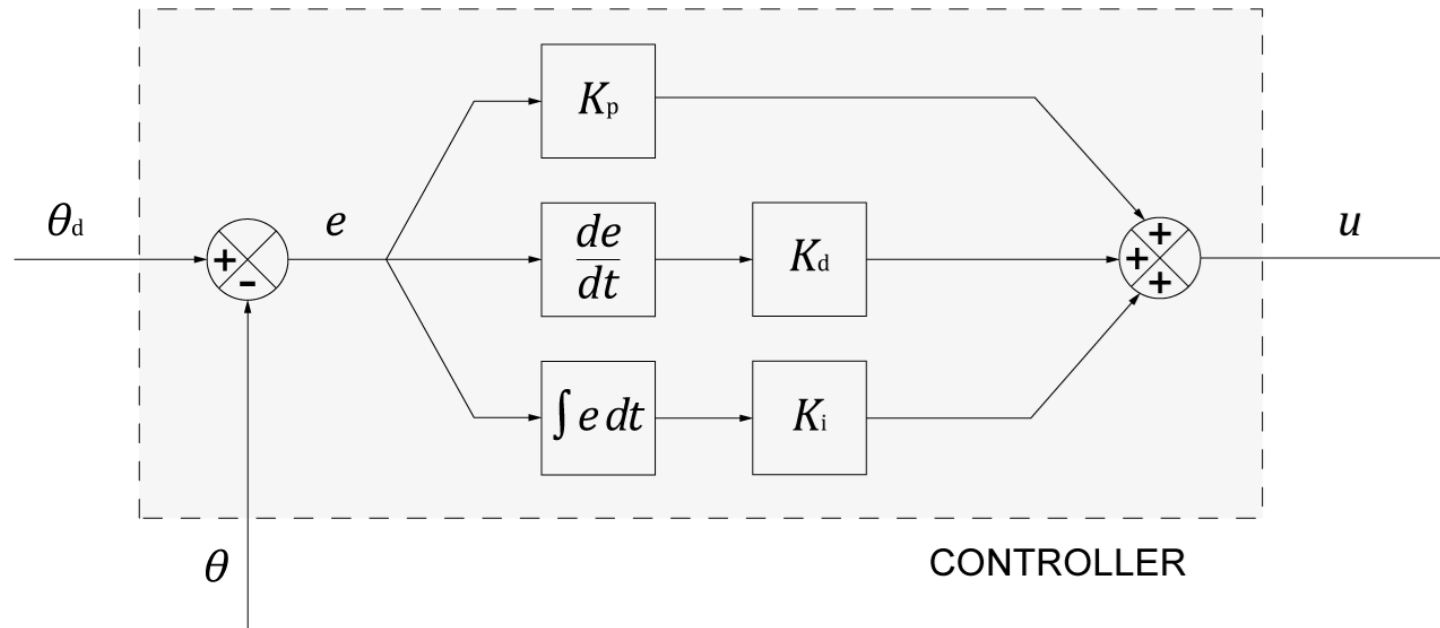
0%

Not really

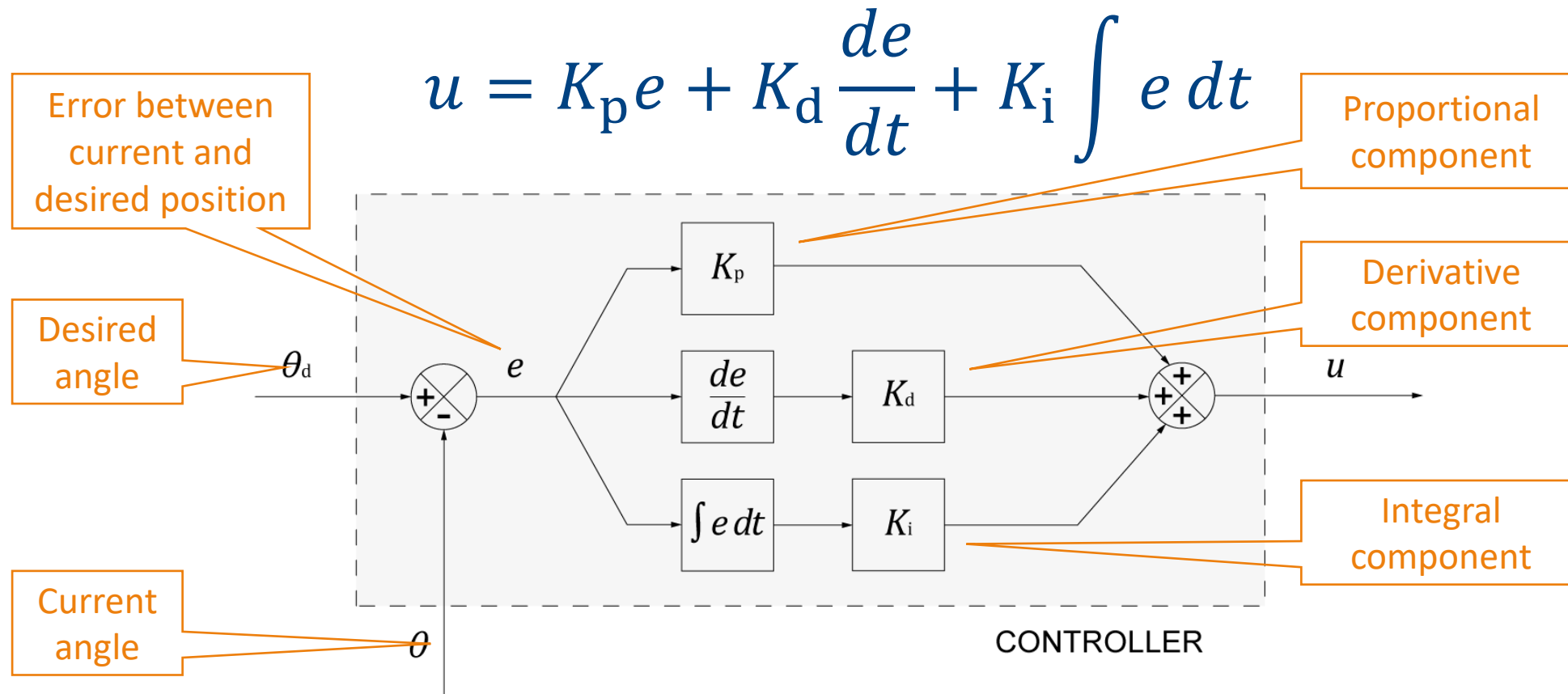
0%

# PID (Proportional, Integral, Derivative) Control

$$u = K_p e + K_d \frac{de}{dt} + K_i \int e dt$$

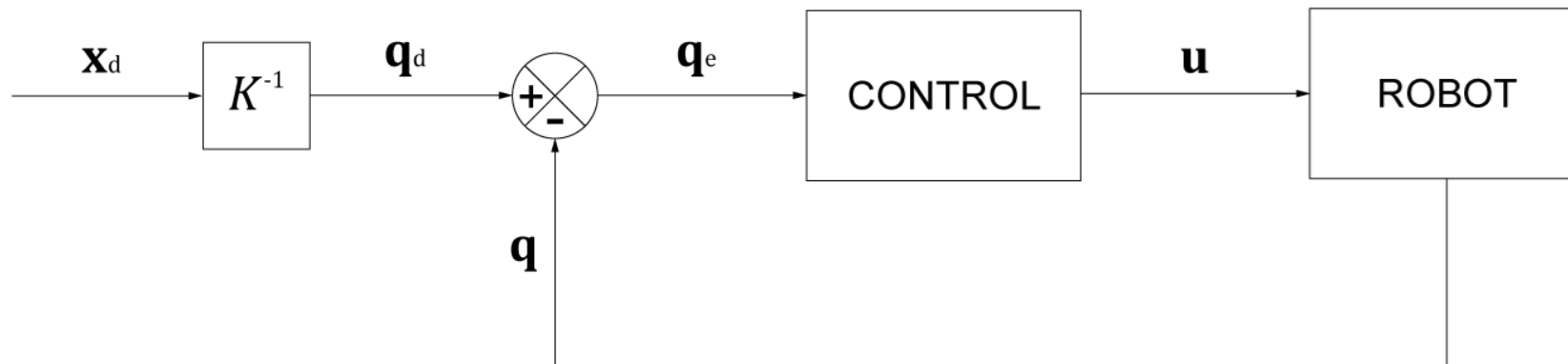


# PID (Proportional, Integral, Derivative) Control



# Joint Space Control\*

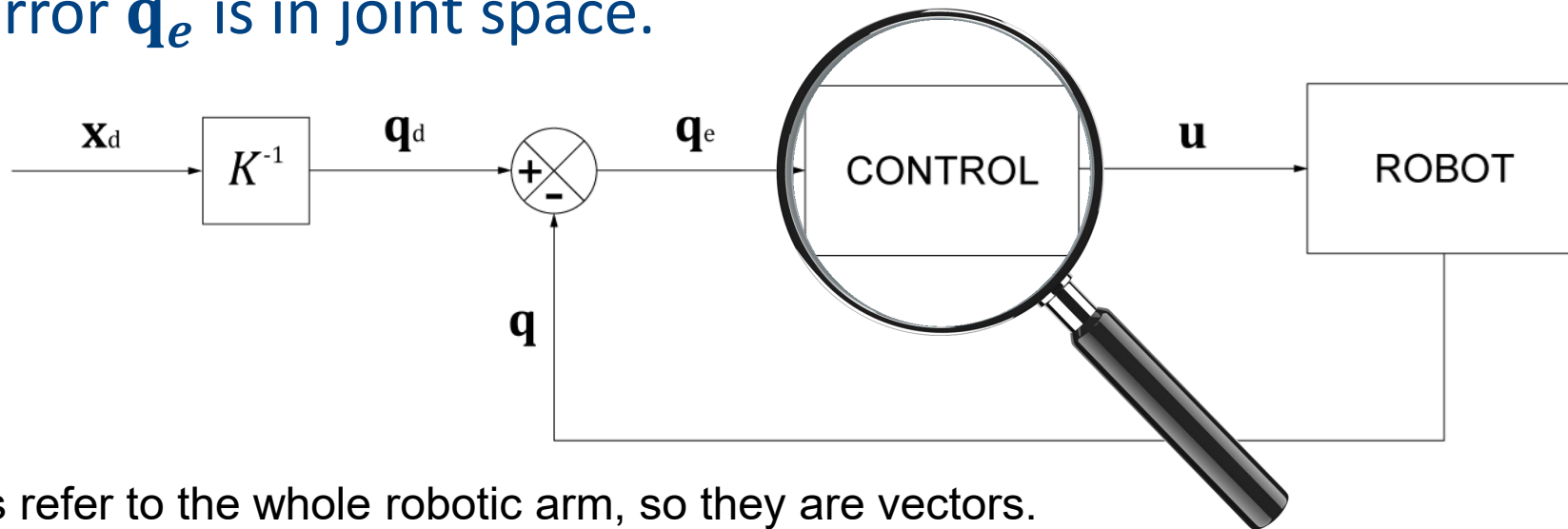
- Inverse kinematic transformation  $K^{-1}$  at the start of the control system, outside of the control loop.
- desired position  $\mathbf{x}_d$  in task space  $\rightarrow$  desired position  $\mathbf{q}_d$  in joint space.
- $\mathbf{q}_d$  is given by position sensors.
- The error  $\mathbf{q}_e$  is in joint space.



\*Positions refer to the whole robotic arm, so they are vectors.

# Joint Space Control\*

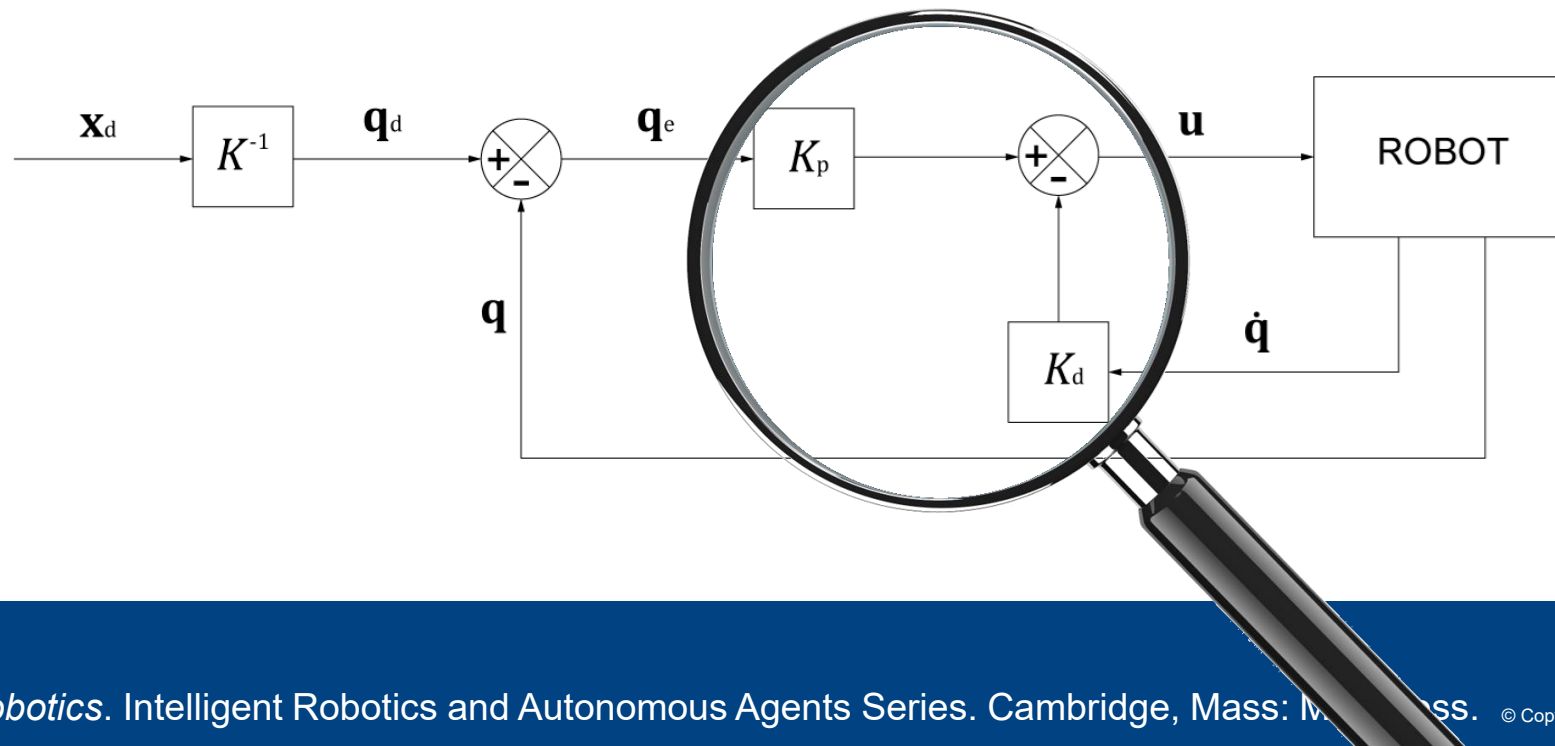
- Inverse kinematic transformation  $K^{-1}$  at the start of the control system, outside of the control loop.
- desired position  $\mathbf{x}_d$  in task space  $\rightarrow$  desired position  $\mathbf{q}_d$  in joint space.
- $\mathbf{q}_d$  is given by position sensors.
- The error  $\mathbf{q}_e$  is in joint space.



\*Positions refer to the whole robotic arm, so they are vectors.

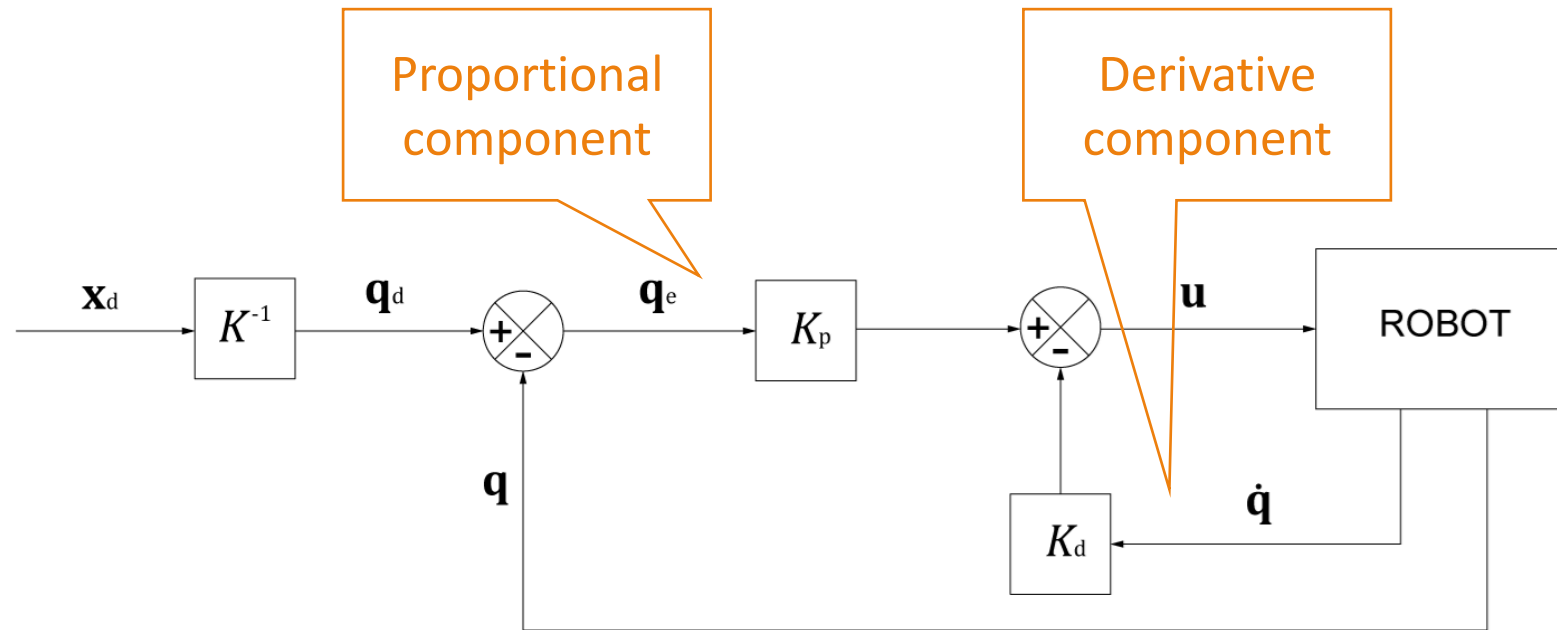
# PD Joint Space Control

$$\mathbf{u} = \mathbf{K}_p \mathbf{q}_e + \mathbf{K}_d \dot{\mathbf{q}}$$



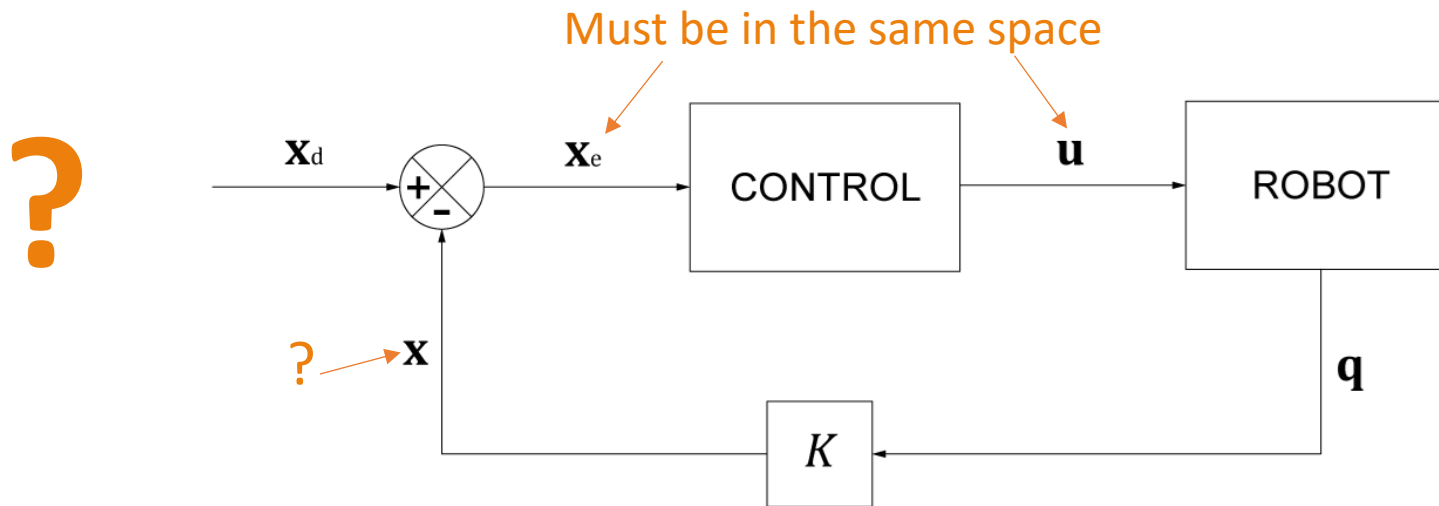
# PD Joint Space Control

$$\mathbf{u} = \mathbf{K}_p \mathbf{q}_e + \mathbf{K}_d \dot{\mathbf{q}}$$



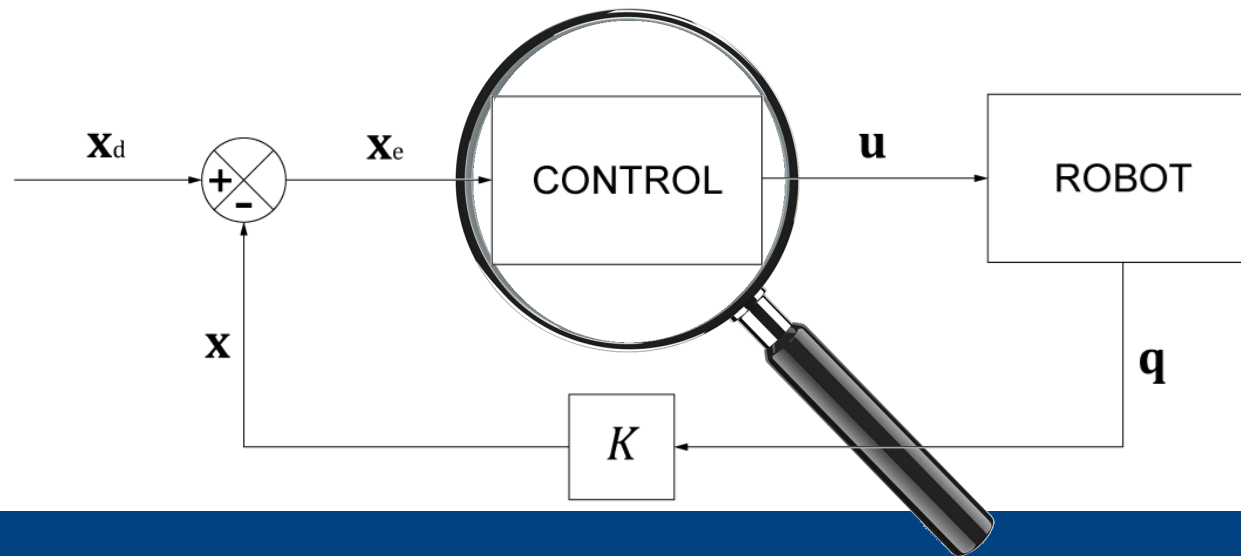
# Task Space Control

- The desired position  $\mathbf{x}_d$  is compared directly with the current position  $\mathbf{x}$ , both in task space.
- $\mathbf{x}$  is obtained by transforming the readings of the position sensors  $\mathbf{q}$  with a forward kinematic transformation
- The error  $\mathbf{x}_e$  is in task space.



# Task Space Control

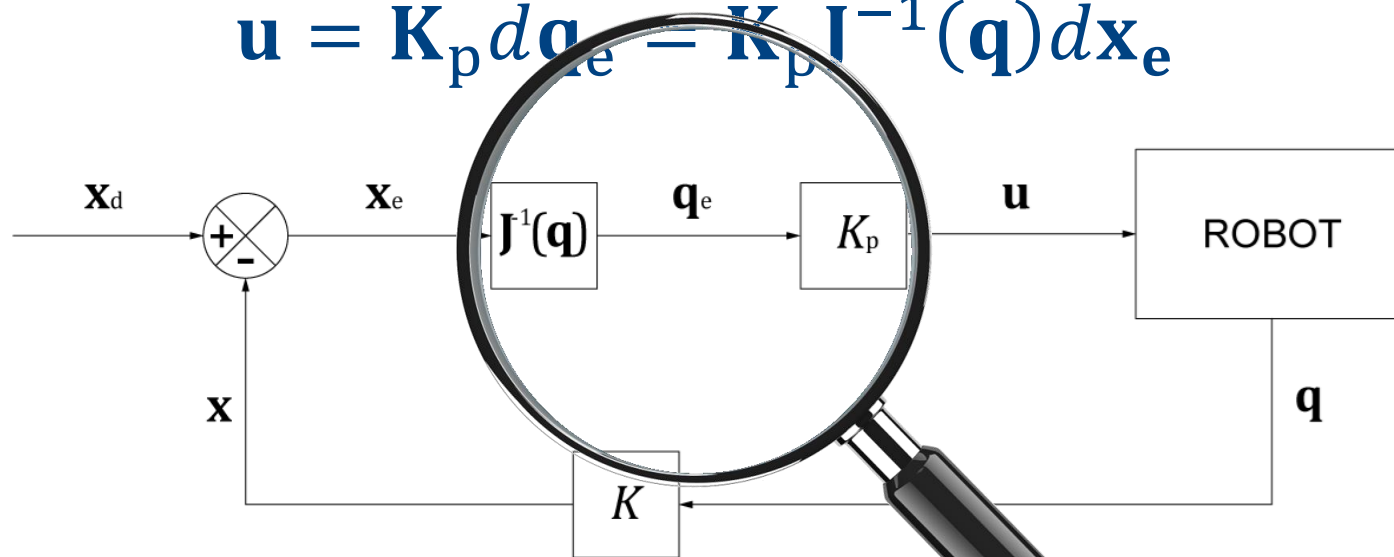
- The desired position  $\mathbf{x}_d$  is compared directly with the current position  $\mathbf{x}$ , both in task space.
- $\mathbf{x}$  is obtained by transforming the readings of the position sensors  $\mathbf{q}$  with a forward kinematic transformation
- The error  $\mathbf{x}_e$  is in task space.



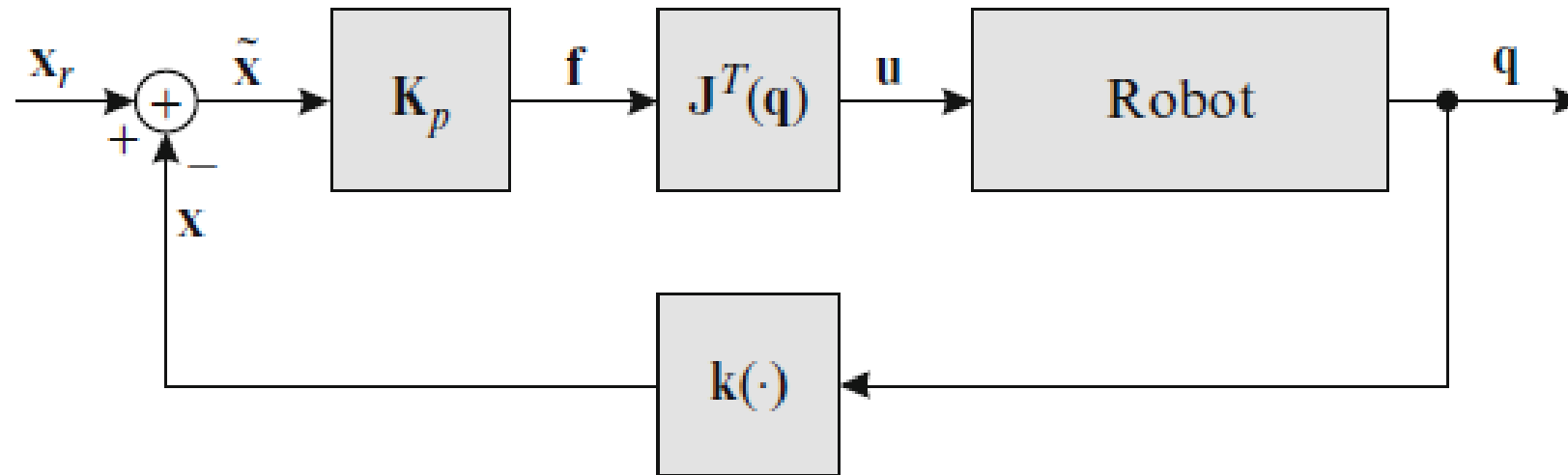
# Control in Task Space Based on Inverse Jacobian

- We know that  $\dot{\mathbf{q}} = \mathbf{J}^{-1}(\mathbf{q})\dot{\mathbf{x}}$ , so we can write  $\frac{d\mathbf{q}}{dt} = \mathbf{J}^{-1}(\mathbf{q})\frac{d\mathbf{x}}{dt}$ .
- For small displacements,  $d\mathbf{q} = \mathbf{J}^{-1}(\mathbf{q})d\mathbf{x}$ .
- The error in task space is transformed into an error in joint space.

$$\mathbf{u} = \mathbf{K}_p d\mathbf{q}_e = \mathbf{K}_p \mathbf{J}^{-1}(\mathbf{q}) d\mathbf{x}_e$$



# Motion control in task space based on the transposed Jacobian matrix



Control based on the transposed Jacobian matrix

$$\mathbf{f} = \mathbf{K}_p \tilde{\mathbf{x}}. \quad \mathbf{u} = \mathbf{J}^T(\mathbf{q}) \mathbf{f}.$$

$\mathbf{f}$  = force at end effector

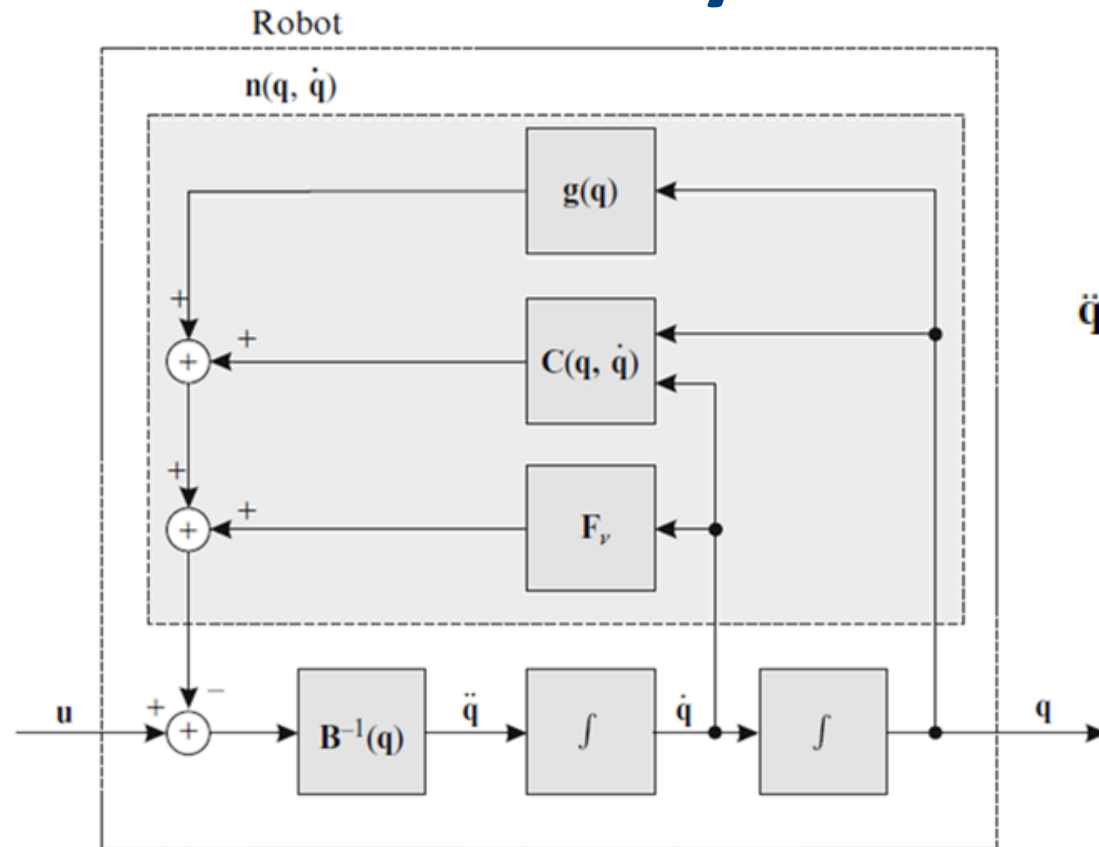
# Robot Dynamic Model

- The torques  $\tau$  to ultimately provide to the actuators are given by the following dynamic model of the robot:

$$\tau = \mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$$

- $\mathbf{B}$  is the inertia matrix, depending on current configuration  $\mathbf{q}$  and multiplied by the joint accelerations  $\ddot{\mathbf{q}}$
- $\mathbf{C}$  is a matrix accounting for Coriolis forces that depend both on the current joint configuration  $\mathbf{q}$  and on the joint velocities  $\dot{\mathbf{q}}$  and is multiplied by the joint velocities  $\dot{\mathbf{q}}$
- $\mathbf{F}$  is the matrix of frictions and multiplies the joint velocities  $\dot{\mathbf{q}}$
- $\mathbf{g}$  accounts for gravity and depends on current robot position

# Motion control in joint space based on inverse dynamics



The direct dynamic model of a robot mechanism  
Robot motion under the influence of given joint torques

$$B(q)\ddot{q} + C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q) = u.$$

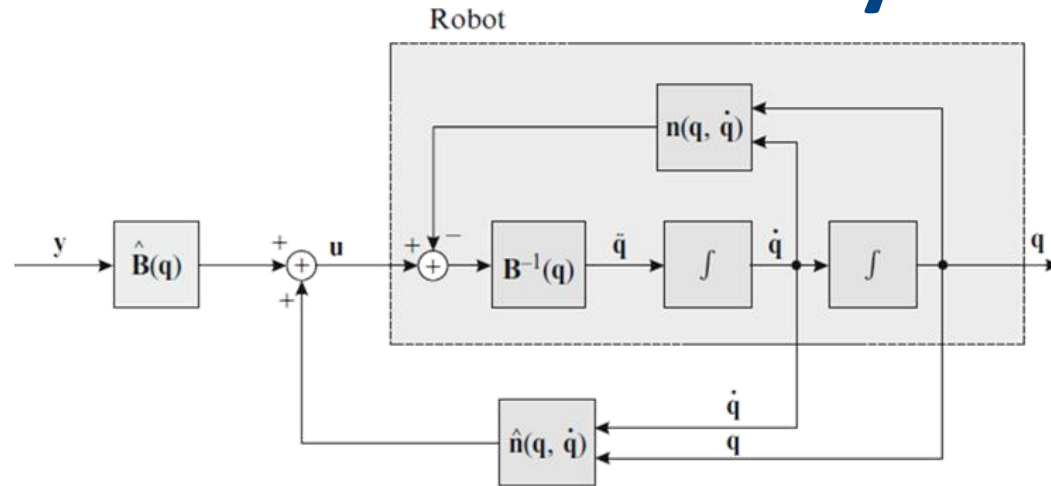
$$\ddot{q} = B^{-1}(q) (u - (C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q))).$$

$$n(q, \dot{q}) = C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q).$$

$$B(q)\ddot{q} + n(q, \dot{q}) = \tau.$$

$$\ddot{q} = B^{-1}(q) (u - n(q, \dot{q})).$$

# Motion control in joint space based on inverse dynamics



$$\mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}_v\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}.$$

$$\ddot{\mathbf{q}} = \mathbf{B}^{-1}(\mathbf{q}) (\mathbf{u} - (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}_v\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}))).$$

## 7.7 Linearization of the control system by implementing the inverse dynamic model

Let us assume that the robot dynamic model is known. The inertial matrix  $\hat{\mathbf{B}}(\mathbf{q})$  is an approximation of the real values  $\mathbf{B}(\mathbf{q})$ , while  $\hat{\mathbf{n}}(\mathbf{q}, \dot{\mathbf{q}})$  represents an approximation of  $\mathbf{n}(\mathbf{q}, \dot{\mathbf{q}})$  as follows

$$\hat{\mathbf{n}}(\mathbf{q}, \dot{\mathbf{q}}) = \hat{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \hat{\mathbf{F}}_v\dot{\mathbf{q}} + \hat{\mathbf{g}}(\mathbf{q}).$$

The controller output  $\mathbf{u}$  is determined by the following equation

$$\mathbf{u} = \hat{\mathbf{B}}(\mathbf{q})\ddot{\mathbf{y}} + \hat{\mathbf{n}}(\mathbf{q}, \dot{\mathbf{q}}),$$

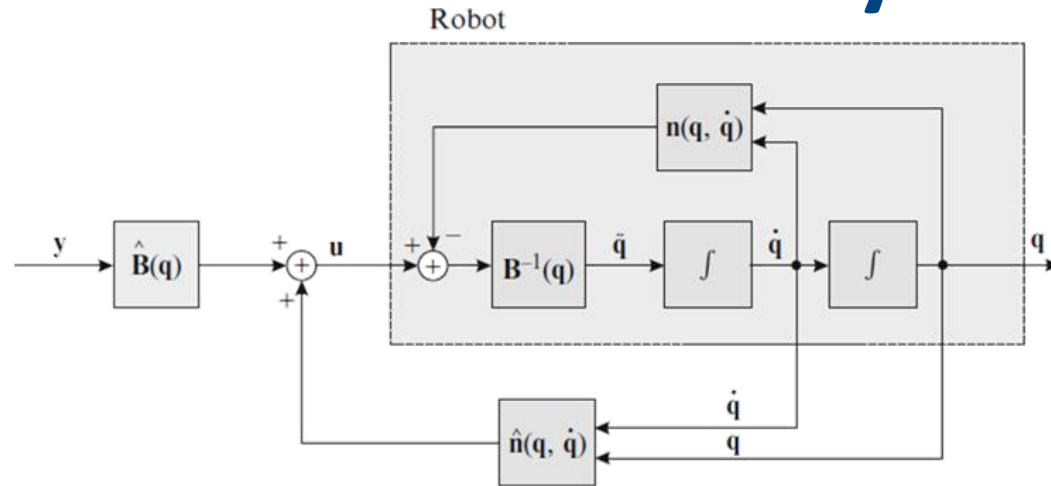
where the approximate inverse dynamic model of the robot was used.

$$\mathbf{n}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}_v\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}).$$

$$\mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{n}(\mathbf{q}, \dot{\mathbf{q}}) = \boldsymbol{\tau}.$$

$$\ddot{\mathbf{q}} = \mathbf{B}^{-1}(\mathbf{q}) (\boldsymbol{\tau} - \mathbf{n}(\mathbf{q}, \dot{\mathbf{q}})).$$

# Motion control in joint space based on inverse dynamics



7.7 Linearization of the control system by implementing the inverse dynamic model

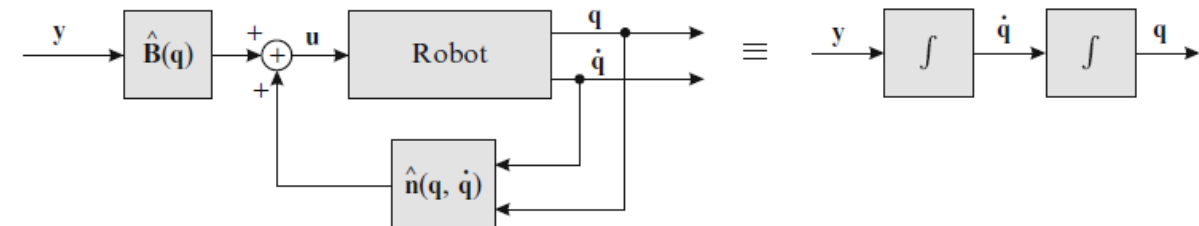
Let us assume that the robot dynamic model is known. The inertial matrix  $\hat{B}(q)$  is an approximation of the real values  $B(q)$ , while  $\hat{n}(q, \dot{q})$  represents an approximation of  $n(q, \dot{q})$  as follows

$$\hat{n}(q, \dot{q}) = \hat{C}(q, \dot{q})\dot{q} + \hat{F}_v\dot{q} + \hat{g}(q).$$

The controller output  $u$  is determined by the following equation

$$u = \hat{B}(q)y + \hat{n}(q, \dot{q}),$$

where the approximate inverse dynamic model of the robot was used.



7.8 The linearized system

# Motion control in joint space based on inverse dynamics

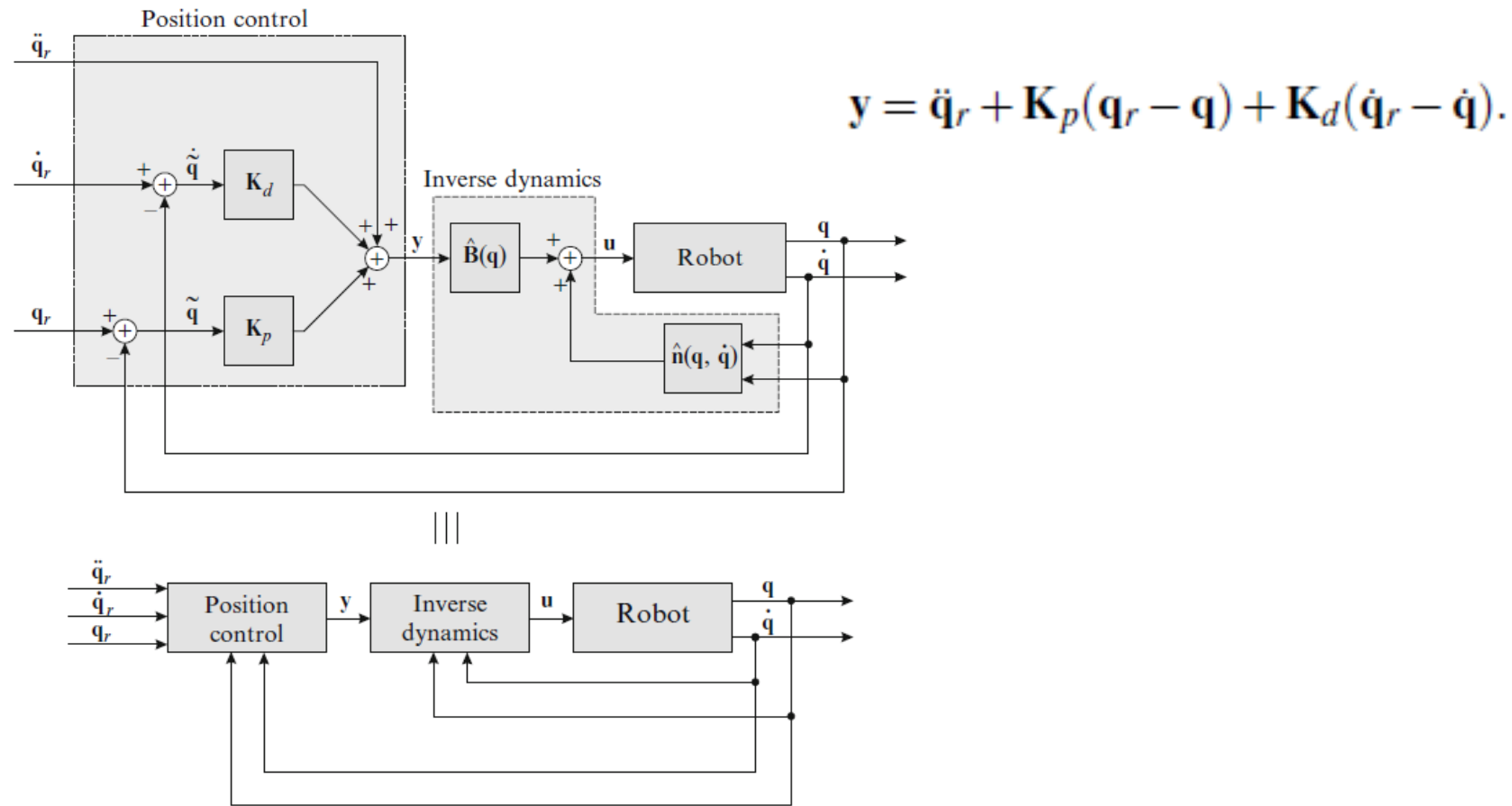


Fig. 7.9 Control of the robot based on inverse dynamics

# Motion control in task space based on inverse dynamics

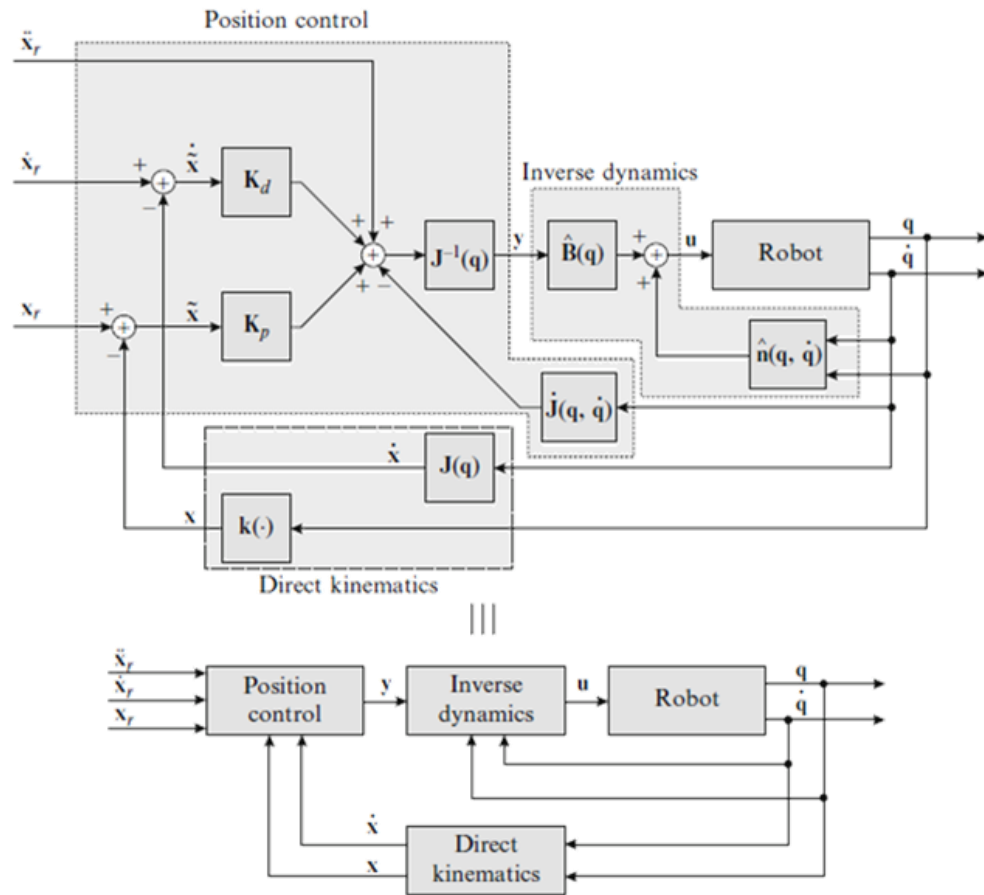


Fig. 7.13 Robot control based on inverse dynamics in external coordinates

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}. \quad (7.33)$$

By calculating the time derivative of equation (7.33), we obtain

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}. \quad (7.34)$$

The error of the pose of the robot end-effector is determined as the difference between its desired and its actual pose

$$\tilde{\mathbf{x}} = \mathbf{x}_r - \mathbf{x} = \mathbf{x}_r - \mathbf{k}(\mathbf{q}). \quad (7.35)$$

In a similar way the velocity error of the robot end-effector is determined

$$\dot{\tilde{\mathbf{x}}} = \dot{\mathbf{x}}_r - \dot{\mathbf{x}} = \dot{\mathbf{x}}_r - \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}. \quad (7.36)$$

The acceleration error is the difference between the desired and the actual acceleration

$$\ddot{\tilde{\mathbf{x}}} = \ddot{\mathbf{x}}_r - \ddot{\mathbf{x}}. \quad (7.37)$$

When developing the inverse dynamics based controller in the internal coordinates, equation (7.19) was derived describing the dynamics of the control error in the form  $\ddot{\tilde{\mathbf{q}}} + \mathbf{K}_d\dot{\tilde{\mathbf{q}}} + \mathbf{K}_p\tilde{\mathbf{q}} = \mathbf{0}$ . An analogous equation can be written for the error of the end-effector pose. From this equation the acceleration  $\ddot{\mathbf{x}}$  of the robot end-effector can be expressed

$$\ddot{\tilde{\mathbf{x}}} + \mathbf{K}_d\dot{\tilde{\mathbf{x}}} + \mathbf{K}_p\tilde{\mathbf{x}} = \mathbf{0} \Rightarrow \ddot{\mathbf{x}} = \ddot{\mathbf{x}}_r + \mathbf{K}_d\dot{\tilde{\mathbf{x}}} + \mathbf{K}_p\tilde{\mathbf{x}}. \quad (7.38)$$

From equation (7.34) we express  $\ddot{\mathbf{q}}$  taking into account the equality  $\mathbf{y} = \ddot{\mathbf{q}}$

$$\mathbf{y} = \mathbf{J}^{-1}(\mathbf{q}) (\ddot{\mathbf{x}} - \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}). \quad (7.39)$$

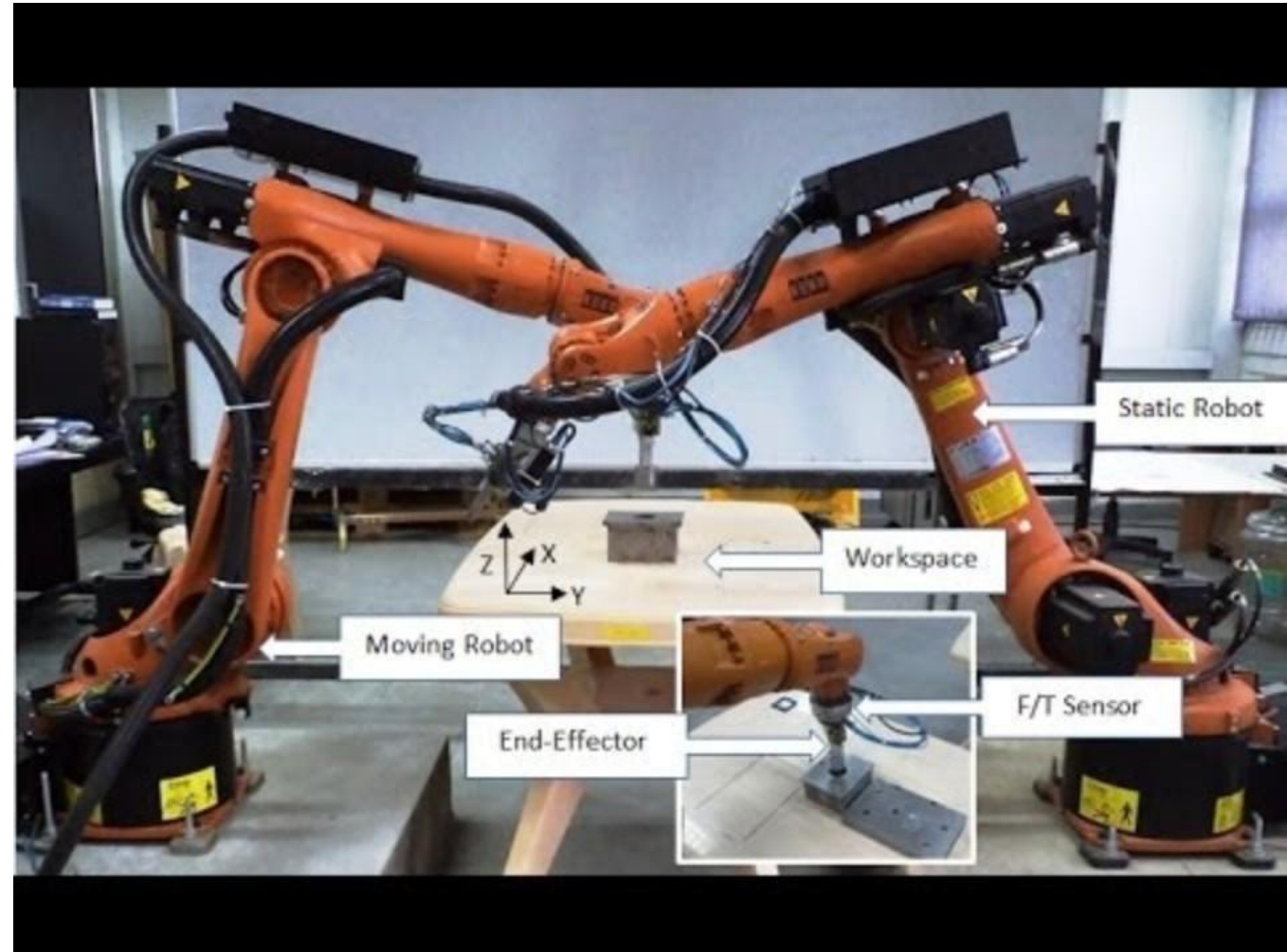
By replacing  $\ddot{\mathbf{x}}$  in equation (7.39) with expression (7.38), the control algorithm based on inverse dynamics in the external coordinates is obtained

$$\mathbf{y} = \mathbf{J}^{-1}(\mathbf{q}) (\ddot{\mathbf{x}}_r + \mathbf{K}_d\dot{\tilde{\mathbf{x}}} + \mathbf{K}_p\tilde{\mathbf{x}} - \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}). \quad (7.40)$$

The control scheme encompassing the linearization of the system based on inverse dynamics (7.31) and the closed loop control (7.40) is shown in Figure 7.13.

# Interaction Control

- **Passive interaction control:** The trajectory of the end-effector is driven by the interaction forces due to the inherent nature or compliance of the robot (i.e., internally, such as joints, servo, joints, etc.).
- In passive control, the end-effector's motion naturally deflects under force, as in soft robots.
- But, this lacks flexibility (every specific task might require a special end-effector to be designed and it can also have position and orientation deviations) and high contact forces could occur because there is no force measurement.



# Interaction Control

- **Active interaction control:** It relies on sensors (e.g., force/torque sensors) and/or feedback controllers to measure interaction forces and adjust the robot's commands accordingly—whether by modifying its trajectory or the way it manipulates objects.
- This approach enables real-time reactions to contact, offering high flexibility and accuracy.
- However, it comes with added complexity and limitations in speed.
- To achieve effective task execution and robust disturbance rejection, active control is typically combined with some degree of passive compliance. Active strategies can be further divided into
  - indirect methods (such as admittance and impedance control)
  - direct force control techniques (such as hybrid force/motion control).



# Robot Dynamic Model

- The torques  $\tau$  to ultimately provide to the actuators are given by the following dynamic model of the robot:

$$\tau = \mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$$

- $\mathbf{B}$  is the inertia matrix, depending on current configuration  $\mathbf{q}$  and multiplied by the joint accelerations  $\ddot{\mathbf{q}}$
- $\mathbf{C}$  is a matrix accounting for Coriolis forces that depend both on the current joint configuration  $\mathbf{q}$  and on the joint velocities  $\dot{\mathbf{q}}$  and is multiplied by the joint velocities  $\dot{\mathbf{q}}$
- $\mathbf{F}$  is the matrix of frictions and multiplies the joint velocities  $\dot{\mathbf{q}}$
- $\mathbf{g}$  accounts for gravity and depends on current robot position

# Robot Dynamic Model with External Forces

- The torques  $\tau$  to ultimately provide to the actuators are given by the following dynamic model of the robot:

$$\tau = \mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q})\mathbf{F}_{tip}$$

- $\mathbf{J}^T(\mathbf{q})$  is the transposed manipulator Jacobian, mapping joint velocities to end-effector velocities
- $\mathbf{F}_{tip}$  is the force and moment vector applied at the end effector (tip)

# Robot Dynamic Model with External Forces in Quasi-static Conditions

$$\tau = \cancel{B(\mathbf{q})\dot{\mathbf{q}}} + \cancel{C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}} + \cancel{F\dot{\mathbf{q}}} + \mathbf{g}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q})\mathbf{F}_{tip}$$

$$\tau = \mathbf{g}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q})\mathbf{F}_{tip}$$

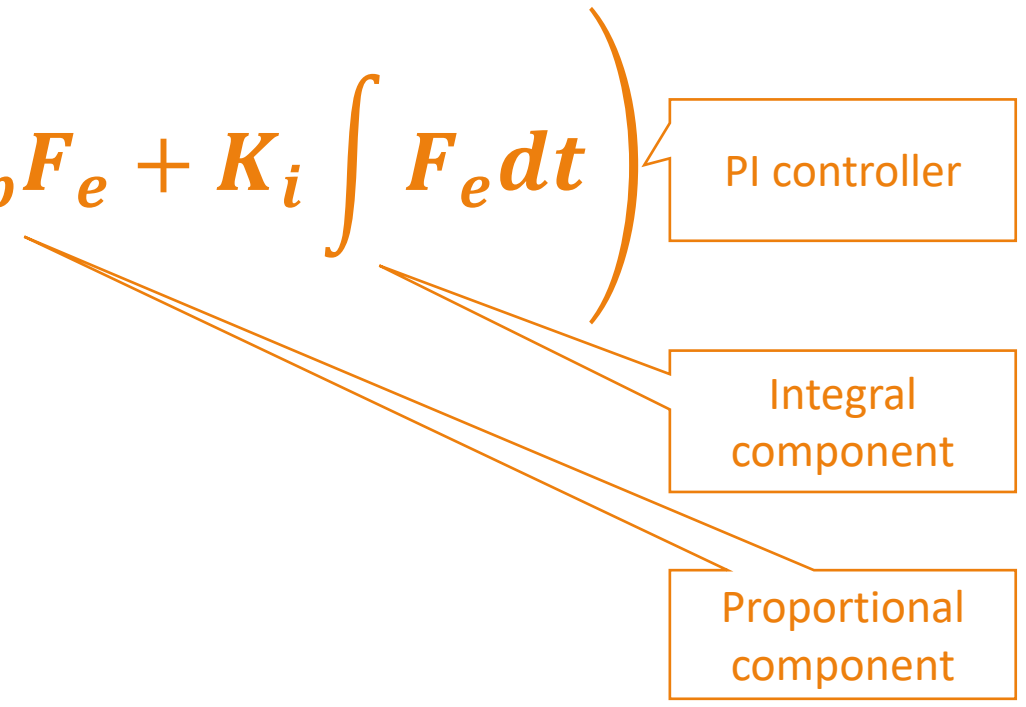
- In **passive interaction control**,  $\mathbf{F}_{tip}$  is imposed by the environment, and the resulting motion depends on the robot's intrinsic compliance.
- In **active interaction control**, this equation becomes a control law: we specify a desired contact force  $\mathbf{F}_d$  and compute the torque needed to apply it:

$$\tau = \tilde{\mathbf{g}}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q})\mathbf{F}_d$$

with  $\tilde{\mathbf{g}}(\mathbf{q})$  an estimation of gravitational torque

# Interaction Control in Quasi-static Conditions

$$\tau = \tilde{\mathbf{g}}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q})\mathbf{F}_d$$

$$\tau = \tilde{\mathbf{g}}(\mathbf{q}) + \mathbf{J}^T(\mathbf{q}) \left( \mathbf{F}_d + \mathbf{K}_p \mathbf{F}_e + \mathbf{K}_i \int \mathbf{F}_e dt \right)$$


PI controller

Integral  
component

Proportional  
component

- $\mathbf{F}_e = \mathbf{F}_d - \mathbf{F}_{tip}$  is the force error
- $\mathbf{K}_p$  is the proportional gain matrix
- $\mathbf{K}_i$  is the integral gain matrix

# Mechanical Impedance

## Mechanical Impedance $Z$ :

how a physical system resists motion when subjected to a force. Unlike static **stiffness** (which links force to displacement), impedance captures **dynamic behavior**, i.e. how velocity  $v$  (or acceleration) influences the force  $F$  a system generates or absorbs.

$$F = Zv$$

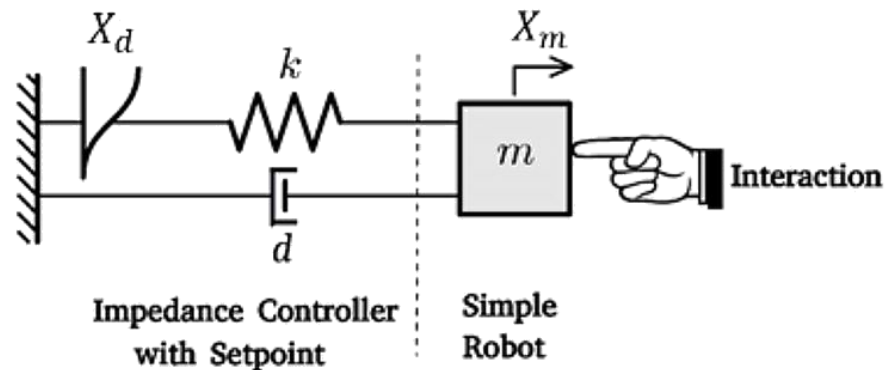
A combination of:

- **Inertial response (mass):**  $F = Ma$
- **Elastic response (Hooke's law):**  $F = Kx$
- **Damping behavior:**  $F = -Bv$

# Impedance Control

$$M_d \ddot{x} + B_d \dot{x} + K_d(x - x_r) = f_{ext}$$

- $x$  is the end-effector position
- $x_r$  is the reference trajectory
- $M_d$  is the desired virtual **mass** matrix
- $B_d$  is the desired virtual **damping** matrix
- $K_d$  is the desired virtual **stiffness** matrix
- $f_{ext}$  is the external force exerted by the environment



Robot as a Spring-Mass-Damper System

- A high stiffness ( $K_d$ ) makes the robot stiffer, resisting deviations from  $x_r$ .
- A high damping ( $B_d$ ) smooths out motion and reduces vibrations.
- A high inertia ( $M_d$ ) makes the robot less sensitive to sudden external disturbances.

# Stiffness Control

## (A Subset of Impedance Control)

- **Static** behaviour
- Instead of specifying how the robot responds dynamically (with mass, damping, and stiffness), we only define a static relationship between the deviation in position/orientation and the force exerted on the environment.
- This is done by acting on the elements of the stiffness **K** in the impedance model while ignoring inertial and damping terms

High stiffness ( $K_d$ )  $\rightarrow$  accurate position tracking, but less compliant

Low stiffness ( $K_d$ )  $\rightarrow$  more compliant, but allows larger motion deviations under external forces.

# Admittance Control (Dual of Impedance)

- In impedance control, we define **how much force** the robot should apply in response to a deviation in motion.
- In admittance control, it's the opposite: we measure an external force and compute **how much the robot should move** to accommodate that force.
  - For instance, if a force of 10N is pushing the robot off its path, an admittance controller might say “yield by 1 mm” (depending on a compliance setting) – effectively, the robot moves slightly until the force reduces.
- Admittance control often involves two loops:
  - an **outer force loop** that modifies the target position based on force input.
  - an **inner high-bandwidth control loop** that ensures accurate tracking of the compliant position or velocity reference.

# Admittance Control

## Outer Loop: Virtual Mechanical Behaviour

$$M_d(\ddot{x}_c - \ddot{x}_r) + B_d(\dot{x}_c - \dot{x}_r) + K_d(x_c - x_r) = f_e$$

- $x_c$  is the compliant position generated by the controller
- $x_r$  is the desired nominal trajectory
- $M_d$  is the desired virtual **mass** matrix
- $B_d$  is the desired virtual **damping** matrix
- $K_d$  is the desired virtual **stiffness** matrix
- $f_e$  is the external force measured at the end-effector

In quasi-static conditions:  $\dot{x}_r = 0$  and  $\ddot{x}_r = 0$ , so:

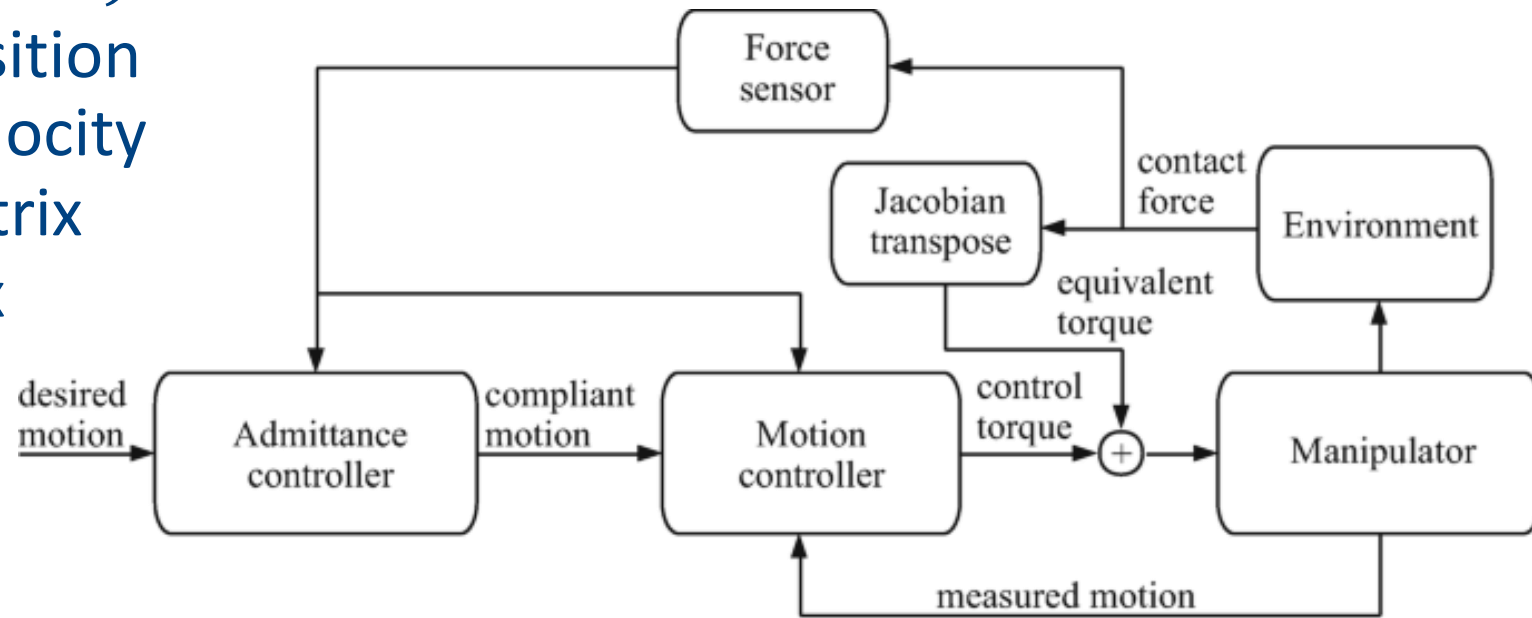
$$M_d\ddot{x}_c + B_d\dot{x}_c + K_d(x_c - x_r) = f_e$$

# Admittance Control

## Inner Loop: Tracking the Compliant Motion

$$F = K_p(x_c - x) + K_d(\dot{x}_c - \dot{x})$$

- $x$ : actual end-effector position
- $\dot{x}$ : actual end-effector velocity
- $K_p$ : proportional gain matrix
- $K_d$ : derivative gain matrix
- $F$ : commanded force or torque to the low-level controller



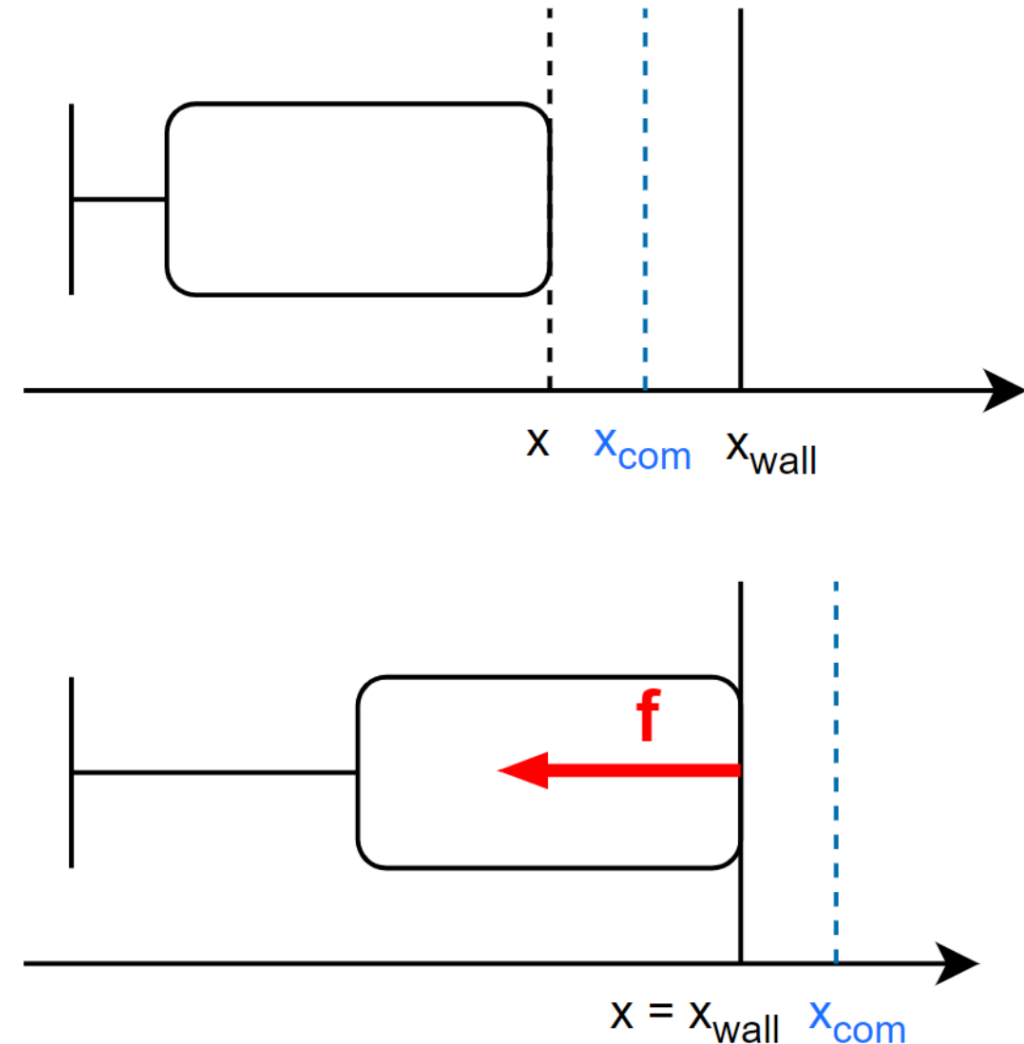
The admittance controller computes a compliant trajectory  $x_c$  based on the external force input  $f_e$  and the desired behavior (mass–spring–damper model). The inner loop tracks  $x_c$  using a motion controller that ensures accurate execution.

# Force Control

- Uses explicit feedback from the interaction with the environment to regulate the forces applied by the robot.
- Based on this contact force, the controller adjusts the robot joint torques or positions to achieve a desired force profile necessary to complete a task.

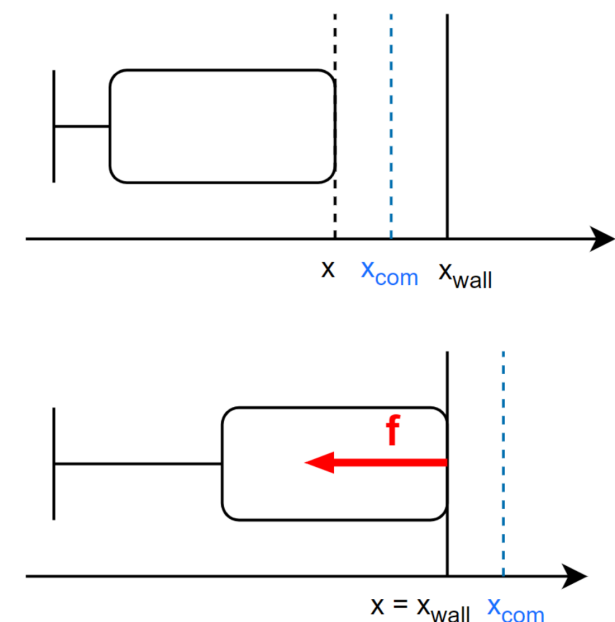
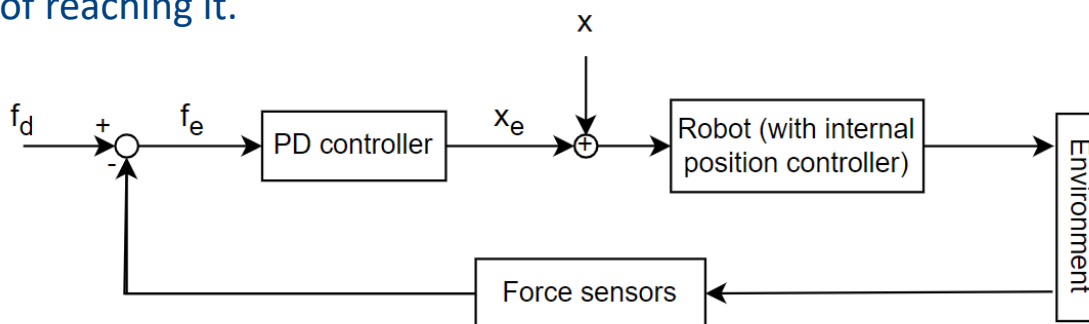
1-DOF, linear, position-controlled robot.

The goal is to control the contact force  $f$  between the end-effector and a wall so that it tracks a desired force  $f_d$ .



# Force Control

- Proportional-Derivative (PD) control law to convert the error  $f_e$  to a position error  $x_e$ .
- Note that the command sent to the robot is a computed desired position  $x_{com} = x + x_e$  and its internal position controller takes care of reaching it.



- When there is no contact, the force/torque sensor measures zero contact force:  $f = 0$ , so the error  $f_e = f_d - f = f_d > 0$ .
- Assuming stationary initial conditions, this creates a position error  $x_e = x_{com} - x > 0$ , which leads to  $x_{com} > 0$  which in turn makes the robot move right toward the wall.
- Upon contact between the robot end-effector and the wall, the sensed force becomes positive but still smaller than  $f_d$  assuming “soft” collision. Therefore, initially, one still has  $x_{com} > x = x_{wall}$  meaning that the robot is commanded to penetrate the wall.
- Finally, interactions between the PD controller and the stiffness of the end-effector, of the wall and of the robot’s internal position controller (characterized in part by its own PID gains) brings the system to a steady state where:
  - $f_e = 0$
  - $x_{com} > x_{wall}$  — i.e. the robot slightly compresses the contact, holding a constant desired force.

# Hybrid Force/Position Control

$$B(q)\ddot{q} + C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q) = \tau - J^T(q)f. \quad \text{Effect of external forces}$$

$$n(q, \dot{q}) = C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q),$$

$$B(q)\ddot{q} + n(q, \dot{q}) = \tau - J^T(q)f.$$

$$B(q)\ddot{q} + n(q, \dot{q}) + J^T(q)f = u.$$

$$\ddot{q} = B^{-1}(q) (u - n(q, \dot{q}) - J^T(q)f).$$

$$u = \hat{B}(q)y + \hat{n}(q, \dot{q}) + J^T(q)f,$$

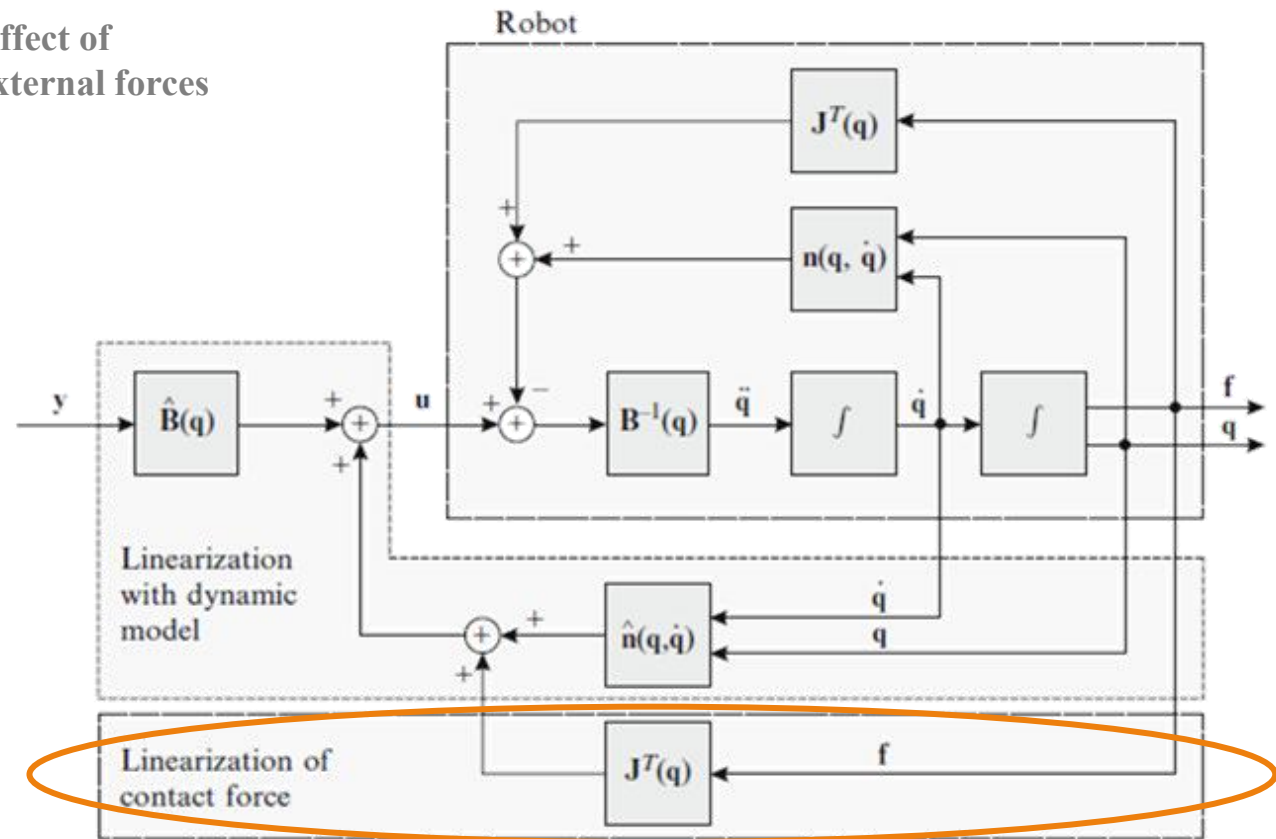


Fig. 7.14 Linearization of the control system by implementing the inverse dynamic model and the measured contact force

# Hybrid Force/Position Control

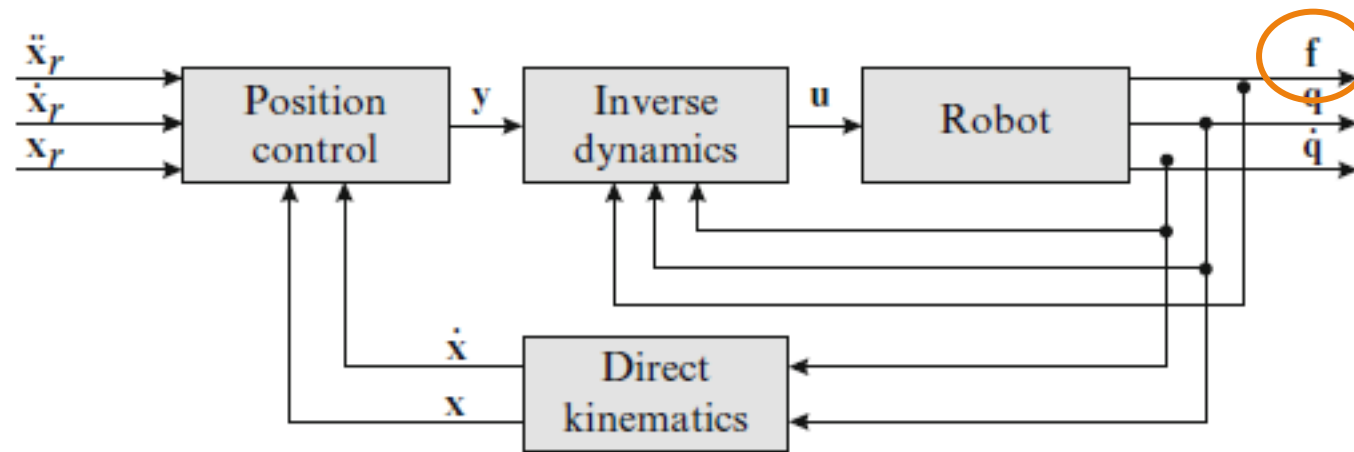
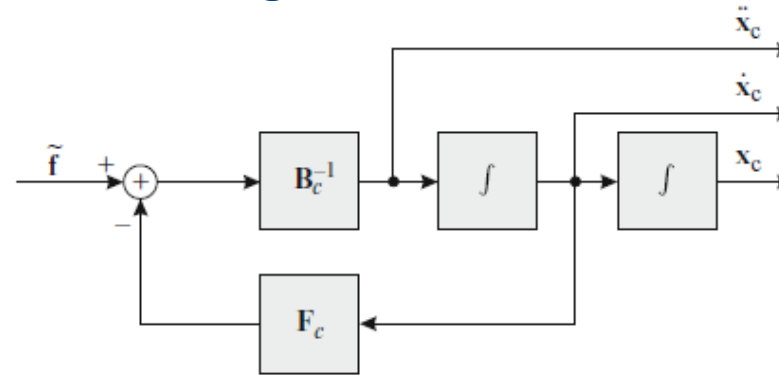


Fig. 7.15 Robot control based on inverse dynamics in external coordinates including the contact force

# Hybrid Force/Position Control



$$\tilde{\mathbf{f}} = \mathbf{B}_c \ddot{\mathbf{x}}_c + \mathbf{F}_c \dot{\mathbf{x}}_c. \quad (7.48)$$

The matrices  $\mathbf{B}_c$  and  $\mathbf{F}_c$  determine the movement of the object under the influence of the force  $\tilde{\mathbf{f}}$ . From equation (7.48) the acceleration of the virtual object can be calculated

$$\ddot{\mathbf{x}}_c = \mathbf{B}_c^{-1} (\tilde{\mathbf{f}} - \mathbf{F}_c \dot{\mathbf{x}}_c). \quad (7.49)$$

$\mathbf{B}_c$  and  $\mathbf{F}_c$  are inertia and damping, respectively, of a virtual object.

Fig. 7.16 Force control translated into control of the pose of robot end-effector

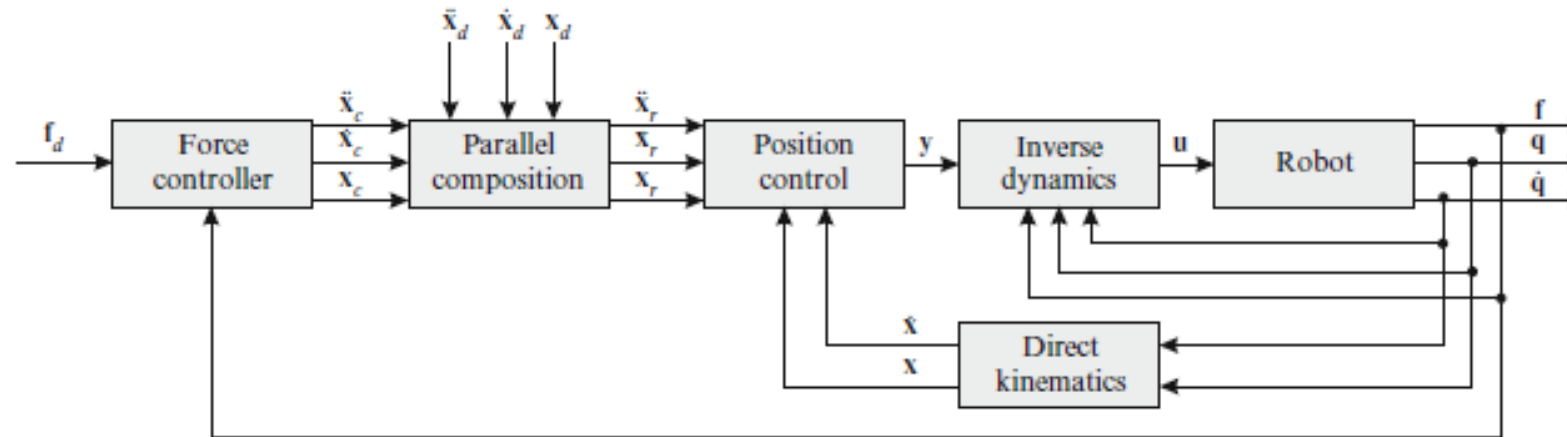


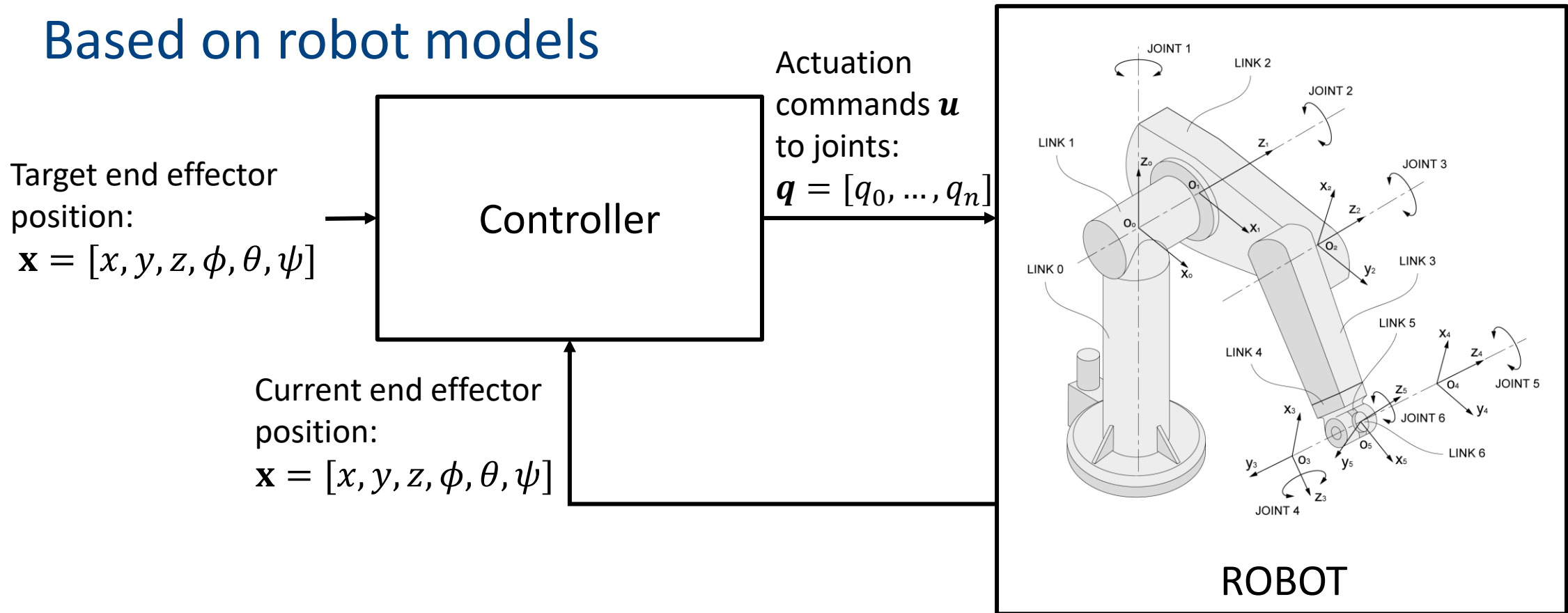
Fig. 7.17 Direct force control in the external coordinates



# Robot Neuro-Controllers

# Robot Controllers

## Based on robot models



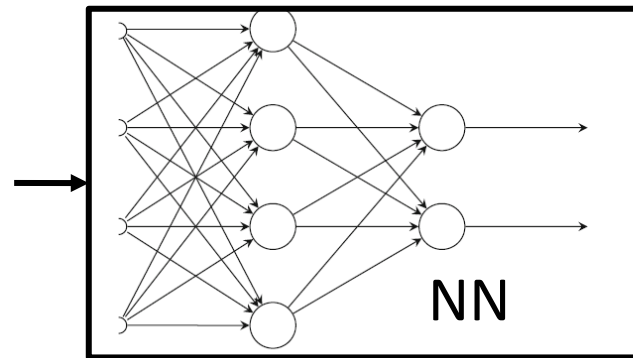
# Robot Neurocontrollers

Target end effector position:

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi]$$

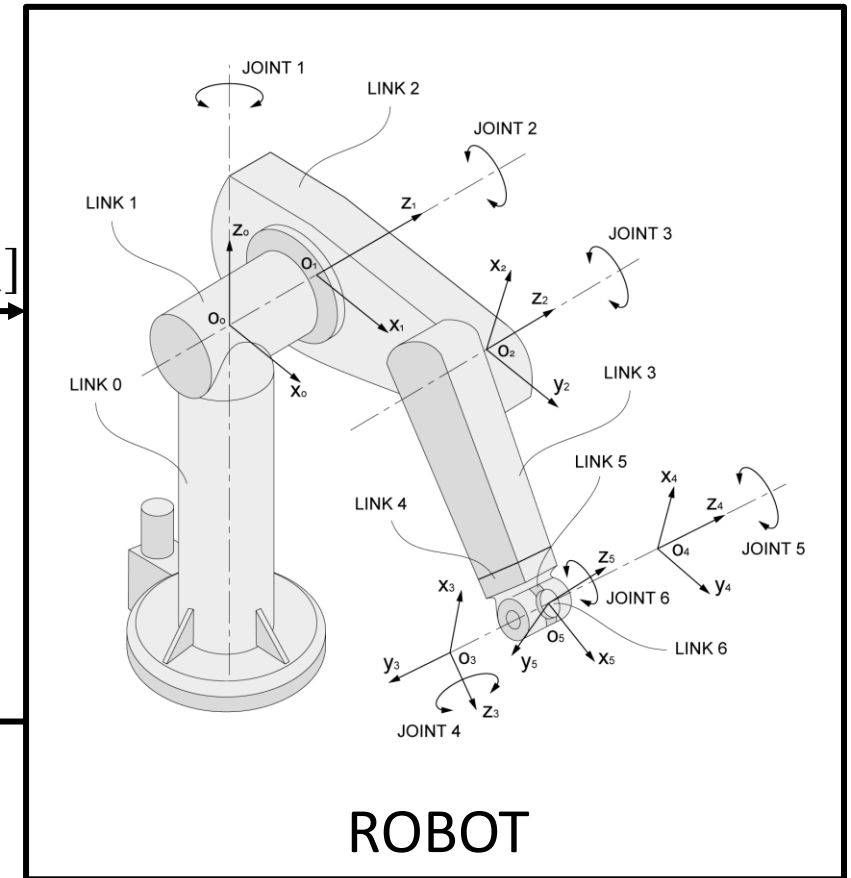
Current end effector position:

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi]$$



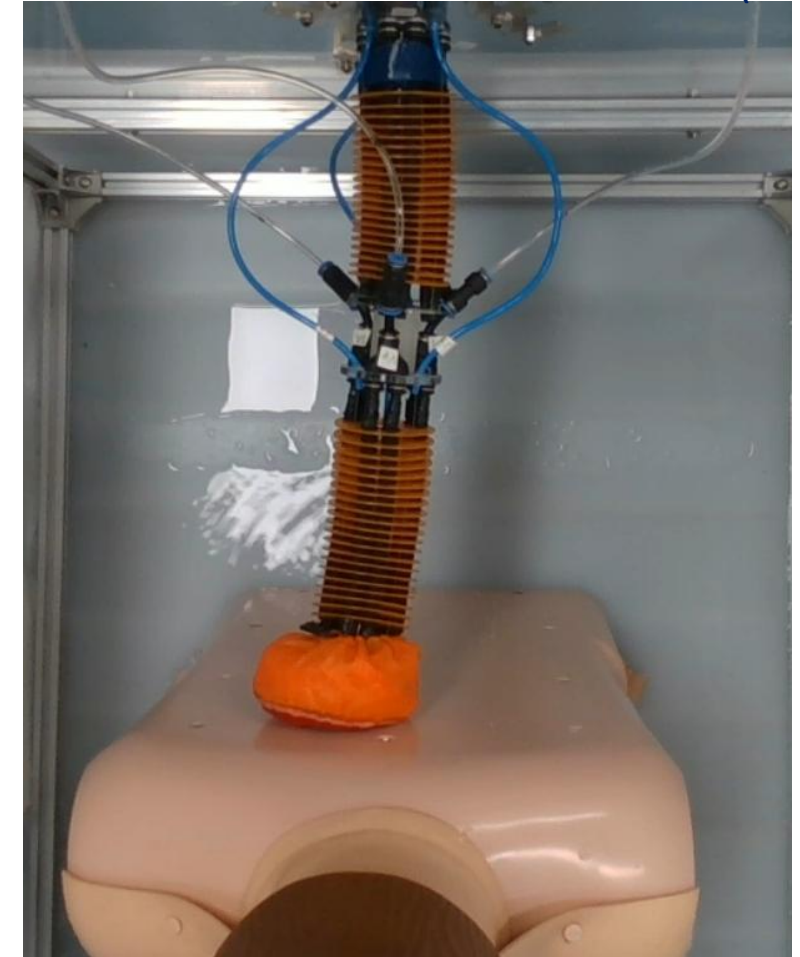
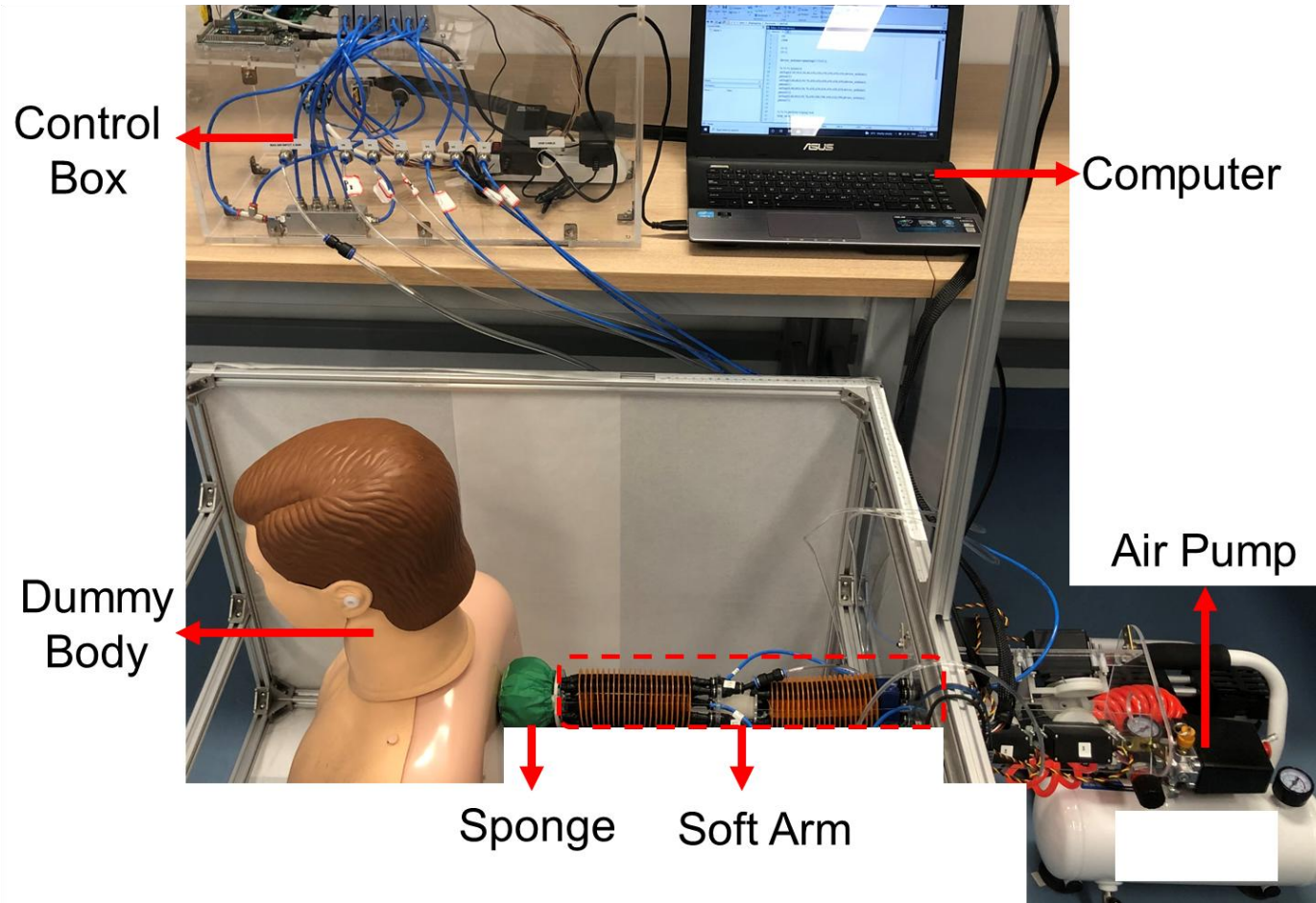
Actuation commands  $\mathbf{u}$  to joints:

$$\mathbf{q} = [q_0, \dots, q_n]$$



- No robot models
- **Learning** used to build the relations between joint space and task space
- **Neural Network (NN)** as controller

# Showering Task Control

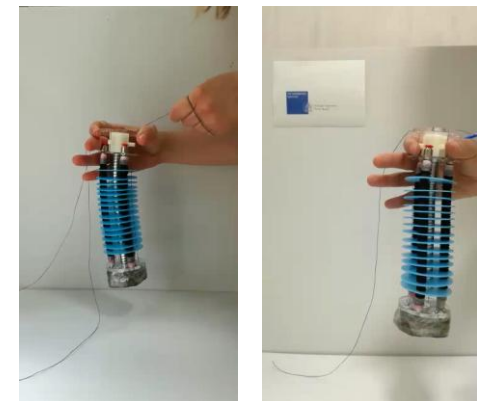
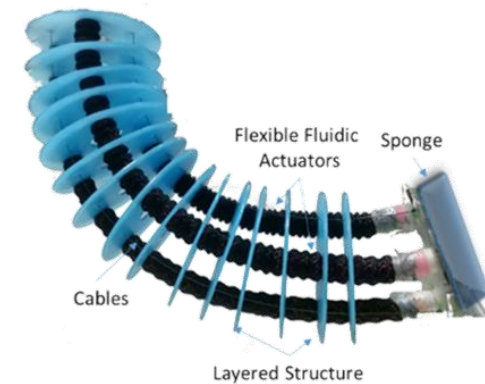


# Position/Force Control of Soft Robot Arm

## Summary

This video demonstrates that a soft robotic manipulator was controlled to wipe the back of a manikin along an inclined path by varying forces.

## Stiffness control





# Non-conventional actuators

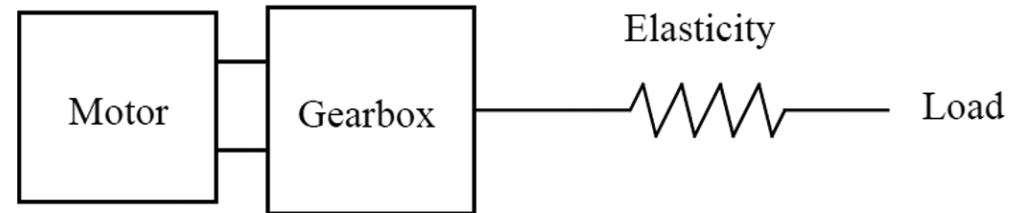
Elastic actuators

# Elastic Actuators

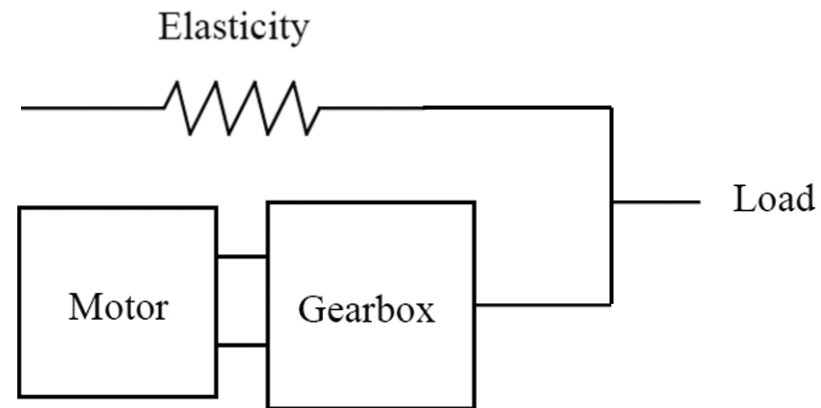
- **Industrial robotics** (structured environment): actuators to be as stiff as possible, to increase accuracy and controllability
- **Service robotics** (unstructured environment): compliance can provide adaptability
- An elastic actuator contains an **elastic element**:
  - tolerance to impact loads,
  - low mechanical output impedance,
  - passive mechanical energy storage,
  - increased peak power output.

# Elastic Actuators – Arrangement

- Serial



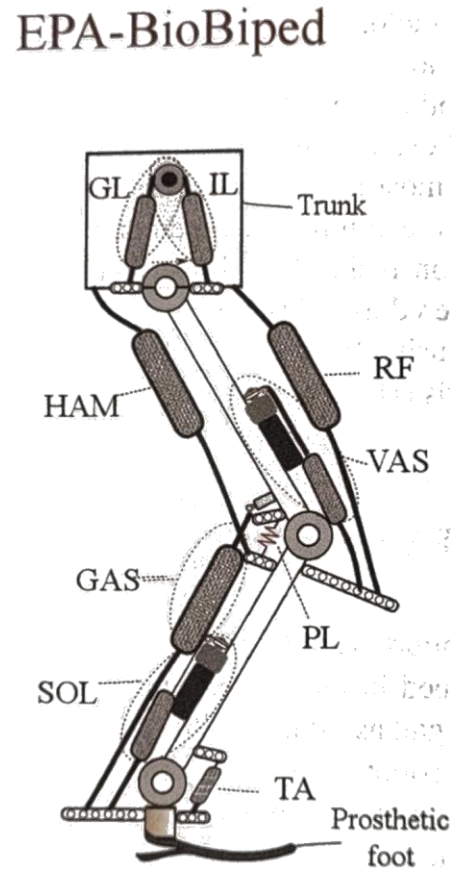
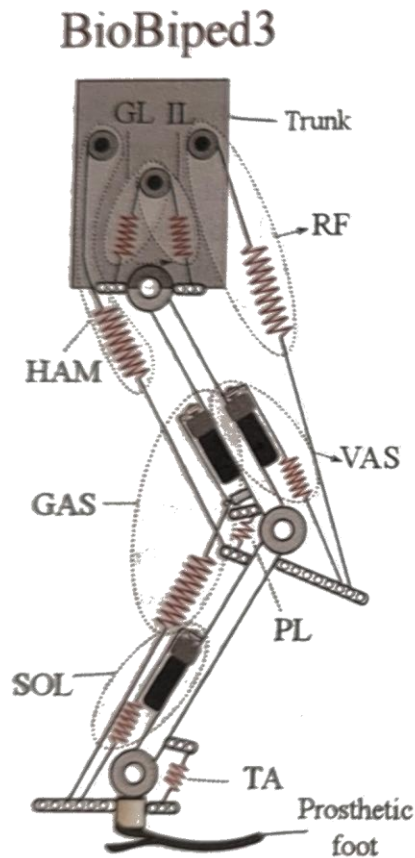
- Parallel



# Series Elastic Actuators – SEA



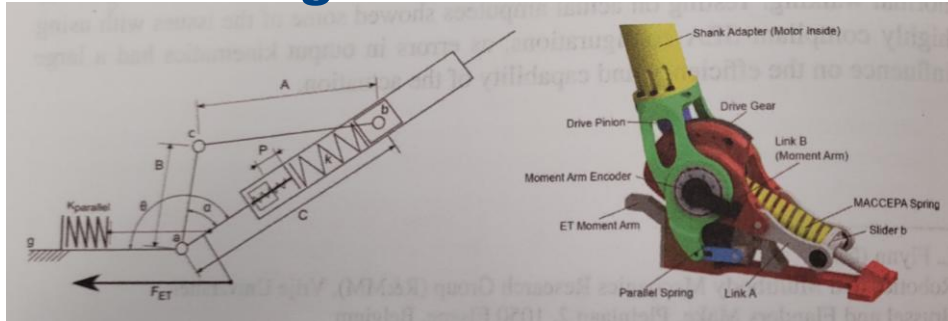
# Hybrid Electric-Pneumatic SEA



- SEA are replaced with EPA (Electric-Pneumatic Actuators)
- Passive springs are replaced with PAM (Pneumatic Artificial Muscles)

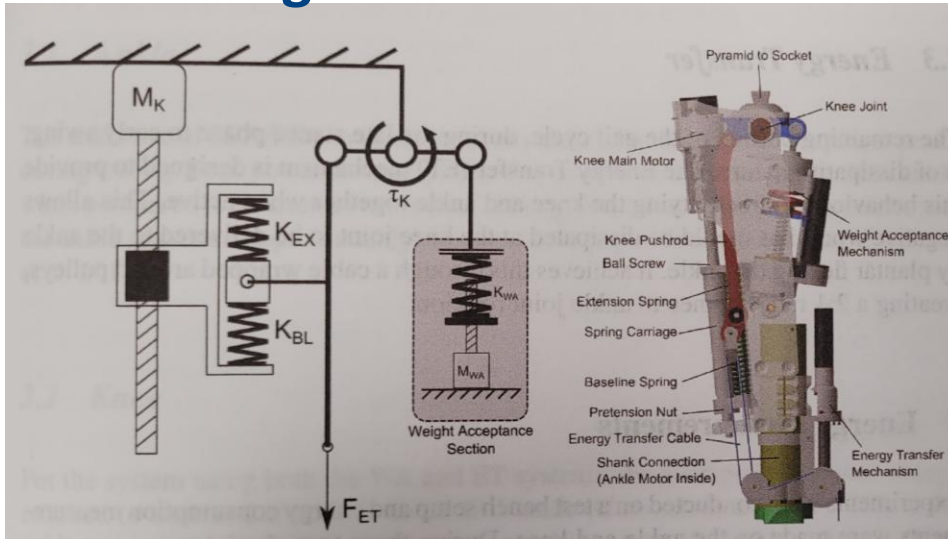
# SEA-PEA example: CYBERLEG prosthesis

## Ankle design



- A main motive actuator
- Series elastic linkage
- Fixed parallel spring

## Knee design



- Main SEA actuator
- Parallel Weight Acceptance actuator (actuated spring)



[https://www.youtube.com/watch?v=a0BEv-wa36Q&ab\\_channel=CYBERLEGS](https://www.youtube.com/watch?v=a0BEv-wa36Q&ab_channel=CYBERLEGS)

# Summary

Questions ?

- Introduction to interaction control
- Strategies for controlling interaction in a robot:
  - Impedance/stiffness/admittance/force control
  - Learning-based control
  - Elastic actuators